

UNIVERSIDADE FEDERAL DE ALAGOAS
INSTITUTO DE COMPUTAÇÃO
CIÊNCIA DA COMPUTAÇÃO

Relatório de Projeto — Compiladores

Compilador da Linguagem MiniPar 2026.2

Orientada a Objetos, com Paralelismo Real (`pure`/`impure`, `fork`/`join`), Execução em Computadores Diferentes e Segunda Interface Web Didática

Professor: Arturo Hernandez Dominguez

Davi Silva de Melo Lins
Iury Kauann David Nogueira
Ludson Lira de Almeida

Código-fonte, programas de teste e relatório no GitHub:
https://github.com/ludsno/minipar-2026-2

Maceió – AL
11 de novembro de 2026

SUMÁRIO

1. INTRODUÇÃO 3
1.1 Arquitetura geral 3
2. A LINGUAGEM MINIPAR 2026.2 5
2.1 Novidades de 2026.2 5
2.2 Gramática BNF da linguagem MiniPar 2026.2 6
3. ANALISADOR LÉXICO 9
3.1 Pseudocódigo do analisador léxico 9
4. ANALISADOR SINTÁTICO 11
4.1 Pseudocódigo do analisador sintático (parser) 11
5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS 13
5.1 Tratamento de erros e comentários 13
5.2 Pseudocódigo do analisador semântico 13
6. TABELA DE SÍMBOLOS 15
6.1 Pseudocódigo da tabela de símbolos 15
7. INTERFACE DO COMPILADOR DA LINGUAGEM MINIPAR 2026.2 16
7.1 Interface de terminal 16
7.2 Primeira interface Web (desenvolvedor) 16
7.3 Segunda interface Web (didática) 16
8. PRODUCT BACKLOG DO COMPILADOR MINIPAR 2026.2 19
9. SPRINTS BACKLOG DO COMPILADOR MINIPAR 2026.2 20
10. DIAGRAMA DE CASOS DE USO (UML) 21
11. ARQUITETURA UTILIZANDO COMPONENTES DE SOFTWARE (UML) 22
12. MODELAGEM DOS COMPONENTES VIA DIAGRAMA DE CLASSES (UML) 24
12.1 Classes dos nós da AST 24
12.2 Diagramas de sequência 24
13. TESTES DE SOFTWARE DE INTEGRAÇÃO DOS COMPONENTES 26
14. TESTES DOS PROGRAMAS DE EXEMPLO NA LINGUAGEM MINIPAR 2026.2 28
14.1 Teste 1 — Calculadora cliente/servidor (c_channel) 28
14.2 Teste 2 — Fatorial e Fibonacci em paralelo (par) 28
14.3 Teste 3 — Neurônio (classes e objetos) 28
14.4 Teste 4 — Rede neural XOR 29
14.5 Teste 5 — Sistema de recomendação 29
14.6 Teste 6 — Quicksort com classes 29
14.7 Teste 7 — Quicksort não recursivo e código de três endereços 30
14.8 Testes 8, 9 e 10 — algoritmos paralelos com fork e join 30
14.8.1 Benchmarks 32
14.9 Testes adicionais 33
14.10 Tabela de estados dos testes 34
15. IMPLEMENTAÇÃO 36
15.a Tecnologias utilizadas 36
15.a.1 Geração de código intermediário (TAC) 36
15.a.2 Geração de código C++ e do executável 37
15.b Passo a passo das telas de execução dos programas de teste 38
15.c Limitações conhecidas 43
16. CÓDIGO-FONTE, PROGRAMAS DE TESTE E RELATÓRIO NO GITHUB 45
17. LINK DO CÓDIGO-FONTE NA CAPA 46
CONCLUSÃO 47
REFERÊNCIAS 48

1. INTRODUÇÃO

Este relatório descreve o projeto **MiniPar 2026.2**, um compilador para a linguagem MiniPar orientada a objetos, com paralelismo real e execução em computadores diferentes. O compilador é escrito em Python e segue a cadeia clássica de compilação: análise léxica, análise sintática (com construção da Árvore Sintática Abstrata — AST), análise semântica com Tabela de Símbolos, geração de código intermediário de três endereços (TAC), geração de código C++20 e compilação com o `g++` (otimização `-O3`), produzindo o arquivo executável `.exe`.

O projeto parte da base do **MiniPar 2026.1** (`minipar-improvement`) e atende ao enunciado de 2026.2, que acrescenta:

1. as primitivas **`fork`** e **`join`**, com a subtarefa podendo ser alocada em um **núcleo da CPU** ou em um **computador diferente**;
2. os **Testes 8, 9 e 10** (algoritmos paralelos em 4, 5 e 6 núcleos, fornecidos em Python);
3. uma **segunda interface Web, didática**, mantendo o terminal e a primeira interface Web;
4. a modelagem com **Componentes de Software** em UML;
5. e, como pedido específico do professor ao grupo após o seminário sobre o artigo SPar (Löff et al., SBLP'21), as anotações **`pure`** e **`impure`**, testadas com o **Conjunto de Mandelbrot**.

1.1 Arquitetura geral

Componente	Arquivo	Responsabilidade
Entrada CLI	`main.py`	Coordena as fases do compilador; aceita um arquivo `.minipar` como argumento.
Menu terminal	`menu.py`	Lista os testes, compila e executa os binários gerados.
IDE Web (desenvolvedor)	`app.py`, `templates/index.html`, `static/js/script.js`	1ª interface Web (2026.1): editor, abas C++/TAC e terminal.
Interface Web didática	`app.py`, `templates/didatico.html`, `static/js/didatico.js`	2ª interface Web (2026.2): tokens, AST gráfica, semântico, TAC, C++, benchmarks, executar e baixar `.exe`.
Interfaces dos componentes	`src/componentes.py`	Classes abstratas `ILexico`, `ISintatico`, `ISemantico`, `ITabelaSimbolos`, `ITAC`, `ICodeGen`.
Léxico (C_Lexer)	`src/lexer.py`	Transforma caracteres em tokens com linha e coluna.
Sintático (C_Parser)	`src/parser.py`	Parser descendente recursivo; constrói a AST.
AST	`src/ast_nodes.py`	Dataclasses dos nós e conversão da árvore para JSON (`ast_to_dict`).
Semântico e Tabela de Símbolos	`src/semantic.py`	Escopos, tipos, funções, classes, regras de `pure`/`impure` e `fork`.
TAC (C_TAC)	`src/tac.py`	Gera o código intermediário de três endereços.
Gerador C++ (C_CodeGen)	`src/codegen.py`, `src/cpp_template.py`	Traduz a AST para C++20, insere o runtime e compila com `g++ -O3`.

Componente	Arquivo	Responsabilidade
Testes automatizados	`tests/test_integracao_componentes.py`	33 testes de integração dos componentes e dos programas de teste.

2. A LINGUAGEM MINIPAR 2026.2

A linguagem combina sintaxe inspirada em C/Python com orientação a objetos e construções nativas de paralelismo. Comentários começam com `#`.

Recurso	Situação no projeto
Tipos e literais	`number` / `int` (inteiros de 64 bits e reais), `string`, `bool`, `list`, `matrix`, `c_channel` e objetos definidos por `class`.
Controle de fluxo	`if` / `else`, `while`, `do while`, `for` clássico, `for in`, `break`, `continue`, `return`.
Orientação a objetos	`class`, `extends`, `new`, `this`, atributos, métodos e chamadas encadeadas por ponto.
Funções	Declaração, parâmetros, chamada, retorno e recursão.
Blocos	`seq { }` (sequencial) e `par { }` (cada comando vira uma thread).
Paralelismo de dados (novo)	`pure for` (padrão Map: iterações divididas entre os núcleos) e `impure { }` (região crítica protegida por mutex). Também `@pure` e `@impure`.
Paralelismo de tarefas (novo)	`fork [nome] [@nucleo(n) | @computador(ip, porta)] comando` e `join`.
Comunicação	`c_channel(ip, porta)`, `send(canal, msg)` e `receive(canal)` sobre sockets TCP.
Biblioteca nativa	`print`, `input`, `exp`, `random`, `range`, `len`, `to_number`, `parse_ints`, `str`, `obter_tempo`, `simular_trabalho`, `nucleo_atual`, e `append` / `pop` / `size` em listas.

2.1 Novidades de 2026.2

`pure` e `impure` (modelo SPar). Um laço anotado com `pure` declara que suas iterações são independentes. O compilador então o traduz para o padrão *Map* (*parallel for*): as iterações são distribuídas entre as threads, uma por núcleo disponível, com escalonamento dinâmico de blocos. Um trecho com efeito colateral em variável compartilhada deve ficar dentro de `impure`, que o compilador protege com `std::lock_guard<std::mutex>`.

```
var total = 0;
pure for (var i = 0; i < 1000; i = i + 1) {
    var parcial = i * 2;          # calculo independente (puro)
    impure {                      # efeito colateral protegido por mutex
        total = total + parcial;
    }
}
```

`fork` e `join` com alocação. O `fork` dispara uma subtarefa e o `join` espera as subtarefas terminarem. Conforme o enunciado, a subtarefa pode ser alocada em um núcleo da CPU ou em um computador diferente:

```
fork soma @nucleo(0) { r1 = soma_ate(10000); }            # thread fixada no nucleo 0
fork fat  @computador("192.168.0.20", 6201) {             # executada em outro computador
    var saida = c_channel("192.168.0.10", 6213);
    send(saida, fatorial(12));                            # dados so por mensagem
}
join;
```

No `@computador`, o mesmo executável roda nos dois computadores: o computador principal envia o pedido da subtarefa e o computador "trabalhador" (iniciado com `MINIPAR_TRABALHADOR=<porta>`) a executa. Não há memória compartilhada: os dados trafegam por `c_channel`. Em `localhost`, o programa inicia o trabalhador sozinho e avisa na tela que a subtarefa está rodando no mesmo computador, em um processo separado.

2.2 Gramática BNF da linguagem MiniPar 2026.2

A gramática do MiniPar 2026.1 foi revisada e complementada com as produções `<bloco_pure>`, `<bloco_impure>`, `<bloco_fork>`, `<alocacao>` e `<comando_join>`. A versão comentada, com exemplos, está no arquivo `BNF-grammar.md` do repositório.

```
(* ============================================================ *)
(*       GRAMÁTICA BNF — LINGUAGEM MINIPAR 2026.2               *)
(*       Equipe: Ludson Almeida, Davi Melo, Iury Nogueira       *)
(*       UFAL - Compiladores (Prof. Arturo Hernandez)           *)
(* ============================================================ *)


(* ─── ESTRUTURA GLOBAL ─── *)

<programa> ::= { <declaracao_global> | <comando> }

<declaracao_global> ::= <declaracao_classe>
                      | <declaracao_funcao>
                      | <declaracao_variavel>


(* ─── DECLARAÇÕES ─── *)

<declaracao_variavel> ::= "var" <IDENTIFIER> [ "=" <expressao> ] ";"
                        | <tipo> <IDENTIFIER> [ "=" <expressao> ] ";"

<declaracao_funcao>   ::= "func" <IDENTIFIER> "(" [ <lista_parametros> ] ")" <bloco_seq>

<lista_parametros>    ::= <parametro> { "," <parametro> }
<parametro>           ::= [ <tipo> ] <IDENTIFIER>

<declaracao_classe>   ::= "class" <IDENTIFIER> [ "extends" <IDENTIFIER> ]
                          "{" { <membro_classe> } "}"

<membro_classe>       ::= "var" <declaracao_variavel>
                        | "func" <declaracao_funcao>


(* ─── COMANDOS ─── *)

<comando> ::= <declaracao_variavel>
            | <atribuicao>
            | <bloco_seq>
            | <bloco_par>
            | <bloco_pure>
            | <bloco_impure>
            | <bloco_fork>
            | <comando_join>
            | <comando_if>
            | <comando_while>
            | <comando_do_while>
            | <comando_for>
            | <comando_print>
            | <comando_send>
            | <comando_return>
            | <comando_break>
            | <comando_continue>
            | <chamada_funcao> ";"
            | <chamada_metodo> ";"

<atribuicao>        ::= <alvo_atribuicao> "=" <expressao> ";"
<alvo_atribuicao>   ::= <IDENTIFIER>
                      | <expressao_posfixo> "." <IDENTIFIER>
                      | <expressao_posfixo> "[" <expressao> "]"

<comando_print>     ::= "print" "(" [ <expressao> { "," <expressao> } ] ")" ";"
<comando_send>      ::= "send"  "(" <expressao> "," <expressao> ")" ";"
<comando_return>    ::= "return" [ <expressao> ] ";"
<comando_break>     ::= "break" ";"
```

```
<comando_continue>  ::= "continue" ";"
<comando_join>      ::= "join" [ <lista_identificadores> ] ";"

<lista_identificadores> ::= <IDENTIFIER> { "," <IDENTIFIER> }


(* ─── BLOCOS E PARALELISMO ─── *)

<bloco_seq>    ::= "seq" "{" { <comando> } "}"
                 | "{" { <comando> } "}"

<bloco_par>    ::= "par" "{" { <comando> } "}"

<bloco_pure>   ::= ( "pure" | "@pure" ) <comando>

<bloco_impure> ::= ( "impure" | "@impure" ) "{" { <comando> } "}"

<bloco_fork>   ::= "fork" [ <IDENTIFIER> ] [ <alocacao> ] <comando>

<alocacao>     ::= "@nucleo" "(" <expressao> ")"
                 | "@computador" "(" <expressao> [ "," <expressao> ] ")"
                 | "(" <expressao> ")"


(* ─── ESTRUTURAS DE CONTROLE ─── *)

<comando_if>       ::= "if" "(" <expressao> ")" <comando> [ "else" <comando> ]

<comando_while>    ::= "while" "(" <expressao> ")" <comando>

<comando_do_while> ::= "do" <comando> "while" "(" <expressao> ")" [ ";" ]

<comando_for>      ::= "for" "(" <for_cabecalho> ")" <comando>

<for_cabecalho>    ::= <for_classico>
                     | <for_iterador>

<for_classico>     ::= [ <inicializador_for> ] ";" [ <expressao> ] ";" [ <incremento_for> ]

<inicializador_for> ::= "var"  <IDENTIFIER> [ "=" <expressao> ]
                      | <tipo> <IDENTIFIER> [ "=" <expressao> ]
                      | <alvo_atribuicao> "=" <expressao>
                      | <expressao>

<incremento_for>   ::= <alvo_atribuicao> "=" <expressao>
                     | <expressao>

<for_iterador>     ::= "var" <IDENTIFIER> "in" <expressao>


(* ─── ORIENTAÇÃO A OBJETOS ─── *)

<expressao_new>         ::= "new" <IDENTIFIER> "(" [ <lista_argumentos> ] ")"
<acesso_propriedade>    ::= <expressao_posfixo> "." <IDENTIFIER>
<chamada_metodo>        ::= <expressao_posfixo> "." <IDENTIFIER> "(" [ <lista_argumentos> ] ")"
<atrib_propriedade>     ::= <expressao_posfixo> "." <IDENTIFIER> "=" <expressao> ";"
<expressao_this>        ::= "this"


(* ─── COMUNICAÇÃO POR CANAIS ─── *)

<expressao_c_channel>   ::= "c_channel" "(" <expressao> "," <expressao> ")"
<expressao_receive>     ::= "receive" "(" <expressao> ")"
(* send já listado em <comando_send> *)


(* ─── COLEÇÕES ─── *)

<expressao_lista>   ::= "[" [ <expressao> { "," <expressao> } ] "]"
<expressao_indice>  ::= <expressao_posfixo> "[" <expressao> "]"
<atrib_indice>      ::= <expressao_posfixo> "[" <expressao> "]" "=" <expressao> ";"
<expressao_matrix>  ::= "matrix" "(" <expressao> "," <expressao> "," <expressao> ")"
```

```
(* ─── ENTRADA E SAÍDA ─── *)

<expressao_input>   ::= "input" "(" [ <expressao> ] ")"
(* print já listado em <comando_print> *)


(* ─── EXPRESSÕES (hierarquia de precedência) ─── *)

<expressao>          ::= <expressao_ou>

<expressao_ou>       ::= <expressao_e> { "||" <expressao_e> }

<expressao_e>        ::= <expressao_comp> { "&&" <expressao_comp> }

<expressao_comp>     ::= <expressao_termo>
                         { ( "==" | "!=" | "<" | ">" | "<=" | ">=" )
                           <expressao_termo> }

<expressao_termo>    ::= <expressao_fator> { ( "+" | "-" ) <expressao_fator> }

<expressao_fator>    ::= <expressao_unaria> { ( "*" | "/" | "%" ) <expressao_unaria> }

<expressao_unaria>   ::= ( "-" | "+" | "!" ) <expressao_unaria>
                       | <expressao_posfixo>

<expressao_posfixo>  ::= <expressao_primaria>
                         { "." <IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ]
                         | "[" <expressao> "]"
                         }

<expressao_primaria> ::= <NUMBER>
                       | <STRING_LITERAL>
                       | <BOOL_LITERAL>
                       | <IDENTIFIER> [ "(" [ <lista_argumentos> ] ")" ]
                       | "this"
                       | <expressao_new>
                       | <expressao_c_channel>
                       | <expressao_receive>
                       | <expressao_input>
                       | <expressao_lista>
                       | <expressao_matrix>
                       | "(" <expressao> ")"

<lista_argumentos>   ::= <expressao> { "," <expressao> }
<chamada_funcao>     ::= <IDENTIFIER> "(" [ <lista_argumentos> ] ")"


(* ─── TIPOS E LITERAIS ─── *)

<tipo>           ::= "number" | "int" | "string" | "bool" | "void" | "list" | "dict"

<NUMBER>         ::= <digito> { <digito> } [ "." { <digito> } ]
<STRING_LITERAL> ::= '"' { <caractere_string> } '"'
<BOOL_LITERAL>   ::= "true" | "false"
<IDENTIFIER>     ::= ( <letra> | "_" ) { <letra> | <digito> | "_" }

<caractere_string> ::= <qualquer_char_exceto_aspas_e_barra>
                     | "\n" | "\t" | '\"' | "\\"

<letra>  ::= "A" | ... | "Z" | "a" | ... | "z"
<digito> ::= "0" | ... | "9"


(* ─── COMENTÁRIOS ─── *)

<comentario> ::= "#" { <qualquer_char> } <nova_linha>
(* Descartados pelo lexer — não aparecem na AST *)
```

3. ANALISADOR LÉXICO

O analisador léxico (`src/lexer.py`, classe `Lexer`, que implementa a interface `ILexico`) percorre o código-fonte caractere a caractere, ignora espaços e comentários iniciados por `#`, mantém linha e coluna e produz objetos `Token(tipo, lexema, linha, coluna)`. Palavras reservadas são resolvidas pelo dicionário `KEYWORDS`; as anotações com arroba são resolvidas à parte, de modo que `nucleo` e `computador` continuam livres como nomes de variáveis.

Categoria	Tokens principais
Palavras reservadas	`class`, `extends`, `new`, `func`, `var`, `if`, `else`, `while`, `do`, `for`, `in`, `break`, `continue`, `return`, `seq`, `par`, **`pure`**, **`impure`**, **`fork`**, **`join`**, `this`, `c_channel`, `send`, `receive`, `input`, `print`, `matrix`
Anotações	`@pure`, `@impure`, **`@nucleo`**, **`@computador`**
Tipos e literais	`TYPE` (`number`, `int`, `string`, `bool`, `void`, `list`, `dict`), `NUMBER`, `STRING_LITERAL`, `BOOL_LITERAL`
Operadores	`+ - * / % = == != ! && || < > <= >=`
Delimitadores	`( ) { } [ ] . ; ,`

3.1 Pseudocódigo do analisador léxico

```
Classe Lexer implementa ILexico:
    Atributos: codigo_fonte, posicao = 0, linha = 1, coluna = 1

    Método pular_espacos_e_comentarios():
        Enquanto posicao < tamanho(codigo_fonte):
            Se caractere_atual for espaço em branco: avancar()
            Senão Se caractere_atual == "#":                # comentário via "#"
                Enquanto caractere_atual != "\n" e não for Fim_Do_Arquivo: avancar()
            Senão: Parar

    Método tokenizar():
        tokens = Lista Vazia
        Enquanto posicao < tamanho(codigo_fonte):
            pular_espacos_e_comentarios()
            Se Fim_Do_Arquivo: Parar
            linha_ini, col_ini = linha, coluna

            Se caractere for letra ou "_":
                ident = ler_enquanto_alfanumerico_ou_sublinhado()
                tipo = PALAVRAS_CHAVE.obter(ident, padrão = IDENTIFICADOR)
                tokens.adicionar(Token(tipo, ident, linha_ini, col_ini))

            Senão Se caractere == "@":                      # anotações
                avancar(); nome = ler_enquanto_alfanumerico_ou_sublinhado()
                Se nome for "pure" ou "impure":   tipo = PURE ou IMPURE
                Senão Se nome for "nucleo":       tipo = ALLOC_NUCLEO
                Senão Se nome for "computador":   tipo = ALLOC_COMPUTADOR
                Senão:                            tipo = DESCONHECIDO
                tokens.adicionar(Token(tipo, "@" + nome, linha_ini, col_ini))

            Senão Se caractere for dígito:
                numero = ler_digitos() [ + "." + ler_digitos() ]
                tokens.adicionar(Token(NUMERO, numero, linha_ini, col_ini))

            Senão Se caractere == '"':
                texto = ler_ate_fechar_aspas(tratando \n, \t, \", \\)
                tokens.adicionar(Token(STRING, texto, linha_ini, col_ini))

            Senão:
                simbolo = avancar()
                Se simbolo puder formar par (==, !=, <=, >=, &&, ||): ler o segundo caractere
                tokens.adicionar(Token(tipo_do_operador_ou_DESCONHECIDO, simbolo, ...))
```

```
        tokens.adicionar(Token(FIM_DO_ARQUIVO, "", linha, coluna))
        Retornar tokens
```

4. ANALISADOR SINTÁTICO

O parser (`src/parser.py`, classe `Parser`, interface `ISintatico`) é **descendente recursivo**: há um método por não-terminal da gramática, e cada método traz, como comentário, a produção BNF que implementa. A precedência dos operadores é garantida pela cadeia `ou → e → comparação → termo → fator → unário → primário`. O resultado é uma AST formada por dataclasses (`Program`, `SeqBlock`, `ParBlock`, `PureBlock`, `ImpureBlock`, `ForkBlock`, `JoinStmt`, `ClassDecl`, `FuncDecl`, `ForStmt`, `BinaryExpr`, entre outras).

No `fork`, o nome da tarefa é opcional; para distinguir `fork f();` (chamada de função) de `fork t1(4) { ... }` (nome + alocação), o parser usa um *lookahead*: se depois do `)` vier `;` ou `.`, trata-se de uma chamada.

4.1 Pseudocódigo do analisador sintático (parser)

```
Classe Parser implementa ISintatico:
    Atributos: tokens, posicao = 0

    Método consumir(tipo_esperado, mensagem):
        Se token_atual.tipo == tipo_esperado: Retornar avancar()
        Senão: falhar(linha_atual, mensagem)        # mostra o erro e lança ErroSintatico

    Método analisar_programa():                     # <programa> ::= { <declaracao> | <comando> }
        programa = Novo Program()
        Enquanto token_atual != FIM_DO_ARQUIVO:
            programa.adicionar(analisar_comando())
        Retornar programa

    Método analisar_comando():
        Se "class":            Retornar analisar_classe()
        Se "func":             Retornar analisar_funcao()
        Se "var" ou TIPO:      Retornar analisar_declaracao_variavel()
        Se "seq" ou "{":       Retornar analisar_bloco_seq()
        Se "par":              Retornar analisar_bloco_par()
        Se "pure":             Retornar analisar_bloco_pure()
        Se "impure":           Retornar analisar_bloco_impure()
        Se "fork":             Retornar analisar_bloco_fork()
        Se "join":             Retornar analisar_join()
        Se "if"/"while"/"do"/"for"/"print"/"send"/"return"/"break"/"continue":
                               Retornar o método correspondente
        Senão:
            expr = analisar_expressao()
            Se token_atual == "=":  Retornar analisar_atribuicao(expr)
            Se expr for chamada de função ou método: consumir(";"); Retornar expr
            falhar("Instrução ou expressão isolada inválida")

    Método analisar_bloco_pure():                   # ("pure" | "@pure") <comando>
        Se token_atual == "for": corpo = analisar_for()
        Senão Se "{":            corpo = analisar_bloco_seq()
        Senão:                   corpo = analisar_comando()
        Retornar PureBlock(corpo)

    Método analisar_bloco_impure():                 # ("impure" | "@impure") "{" { <comando> }
"}"
        consumir("{"); comandos = analisar_ate("}"); consumir("}")
        Retornar ImpureBlock(comandos)

    Método analisar_bloco_fork():                   # "fork" [IDENT] [<alocacao>] <comando>
        nome = Nulo; tipo_aloc = Nulo; args = []
        Se token_atual for IDENT e não for uma chamada (lookahead): nome = avancar()
        Se "@nucleo":       tipo_aloc = "nucleo";     args = analisar_argumentos()   # 1
argumento
        Se "@computador":   tipo_aloc = "computador"; args = analisar_argumentos()   # (ip,
porta) ou (canal)
        Se nome e "(":      tipo_aloc = "nucleo";     args = [expressão]             # forma
legada
        corpo = analisar_bloco_seq() se "{" senão analisar_comando()
        Retornar ForkBlock(corpo, nome, tipo_aloc, args)
```

```
Método analisar_join():                          # "join" [ IDENT { "," IDENT } ] ";"
    alvos = ler_identificadores_separados_por_virgula()
    Retornar JoinStmt(alvos)

Método analisar_expressao(): Retornar analisar_ou()
Método analisar_ou():
    esquerda = analisar_e()
    Enquanto token_atual == "||":
        op = avancar(); esquerda = BinaryExpr(op, esquerda, analisar_e())
    Retornar esquerda
# a cadeia desce para: && → comparações → + - → * / % → unário (! - +) → primário/posfixo
```

5. ANALISADOR SEMÂNTICO E TRATAMENTO DE ERROS

O analisador semântico (`src/semantic.py`, classe `SemanticAnalyzer`, interface `ISemantico`) faz duas passadas: primeiro coleta as declarações globais de funções e classes (o que permite recursão e chamadas antes da declaração); depois valida cada nó da AST, preenchendo a Tabela de Símbolos.

Regra verificada	Mensagem de erro
Redeclaração no mesmo escopo	`Variavel 'x' ja declarada.`
Uso antes da declaração	`Variavel 'x' nao declarada.`
Compatibilidade de tipos	`Tipo incompativel em 'n': esperado number, recebido string.`
Funções	`Funcao 'f' nao declarada.` / `Funcao 'f' espera 2 argumento(s), recebeu 1.`
Classes, métodos e atributos	`Classe 'C' nao declarada.` / `Metodo 'm' nao declarado em 'C'.`
Herança	`Superclasse 'S' nao declarada.`
Controle de fluxo	`'break' usado fora de laco.`
SPar: `impure` fora de `pure`	`A anotacao 'impure' so e permitida dentro de um bloco ou laco 'pure'.`
SPar: escrita em variável externa dentro de `pure`, fora de `impure`	`Atribuicao a variavel externa 'x' dentro de 'pure' sem regiao 'impure' (condicao de corrida).`
Fork: subtarefa `@computador` usando variável externa ou `this`	`A subtarefa 't' (fork @computador) nao pode usar a variavel externa 'x': ela executa em outro computador; envie os dados por c_channel.`

5.1 Tratamento de erros e comentários

Erros léxicos viram tokens `UNKNOWN`, reportados pelo parser. Erros sintáticos mostram `[ERRO SINTATICO] Linha N: ...` e lançam `ParseError` (a linha de comando encerra com código 1; a interface web exibe a mensagem ao aluno). Erros semânticos mostram `[ERRO SEMANTICO] Linha N: ...`, e a análise continua para reportar todos de uma vez. Em tempo de execução, `to_number` com texto inválido mostra um aviso e usa 0, em vez de derrubar o programa.

Comentários com `#`. Como pede o enunciado, a linguagem aceita comentários de linha iniciados por `#`: o analisador léxico descarta tudo do `#` até o fim da linha, então comentários nunca chegam ao parser nem geram erro. Todos os programas de teste usam esse recurso nos seus cabeçalhos.

5.2 Pseudocódigo do analisador semântico

```
Classe AnalisadorSemantico implementa ISemantico:
    Atributos:
        tabela: TabelaDeSimbolos
        erros: Lista de Textos
        funcao_atual, classe_atual: Texto = Nulo
        profundidade_laco: Inteiro = 0
        dentro_de_pure, dentro_de_impure: Booleano = Falso
        base_escopo_pure: Inteiro = Nulo      # primeiro escopo criado dentro do pure atual
        base_escopo_remoto: Inteiro = Nulo    # idem para fork @computador

    Método reportar_erro(linha, mensagem):
        Imprimir "[ERRO SEMANTICO] Linha " + linha + ": " + mensagem
        erros.adicionar(mensagem)

    Método inferir_tipo(no):
        Numero → "number"; Texto → "string"; Booleano → "bool"; Lista → "list"
```

```
        Identificador → tipo do símbolo na tabela (ou "unknown")
        ExpressaoBinaria: relacional/lógico → "bool"; "+" com string → "string"; senão "number"
        Demais casos → "unknown" (compatível com qualquer tipo)

    Método validar_no(no):
        DeclaracaoVariavel:
            validar_no(no.valor); tipo = inferir_tipo(no.valor)
            Se tipos incompatíveis: reportar_erro(...)
            Se não tabela.declarar(no.nome, tipo, escopo_atual, no.linha):
                reportar_erro("Variavel ja declarada")
        Atribuicao:
            simbolo = tabela.buscar(no.nome)
            Se simbolo for Nulo: reportar_erro("Variavel nao declarada")
            Senão:
                # Regra SPar: escrita em variável externa só dentro de impure
                Se base_escopo_pure != Nulo e não dentro_de_impure
                   e tabela.profundidade(no.nome) < base_escopo_pure:
                    reportar_erro("Atribuicao a variavel externa ... (condicao de corrida)")
                verificar_acesso_remoto(no.nome)
            validar_no(no.valor)
        Identificador:
            Se tabela.buscar(no.nome) for Nulo: reportar_erro("Variavel nao declarada")
            Senão: verificar_acesso_remoto(no.nome)
        BlocoSeq / BlocoPar: tabela.entrar_escopo(); validar filhos; tabela.sair_escopo()
        While / DoWhile / For: profundidade_laco += 1; validar corpo; profundidade_laco -= 1
        Break / Continue: Se profundidade_laco == 0: reportar_erro("usado fora de laco")
        BlocoPure:
            salvar estado; dentro_de_pure = Verdadeiro
            base_escopo_pure = quantidade_de_escopos(tabela)
            validar_no(no.corpo); restaurar estado
        BlocoImpure:
            Se não dentro_de_pure: reportar_erro("'impure' so e permitida dentro de 'pure'")
            dentro_de_impure = Verdadeiro; validar filhos; restaurar
        BlocoFork:
            validar argumentos da alocação          # avaliados no computador principal
            Se no.alocacao == "computador":
                base_escopo_remoto = quantidade_de_escopos(tabela)
                validar_no(no.corpo); restaurar
            Senão: validar_no(no.corpo)
        Funções, Classes, Chamadas e Métodos: existência, aridade e escopos próprios

    Método verificar_acesso_remoto(nome):
        Se base_escopo_remoto != Nulo e tabela.profundidade(nome) < base_escopo_remoto:
            reportar_erro("A subtarefa (fork @computador) nao pode usar a variavel externa ...")

    Método analisar(programa):
        coletar_declaracoes_de_classes_e_funcoes(programa)
        Para cada instrucao em programa: validar_no(instrucao)
        Retornar (erros está vazia)
```

6. TABELA DE SÍMBOLOS

A Tabela de Símbolos é um componente próprio (`TabelaSimbolos`, interface `ITabelaSimbolos`, em `src/semantic.py`), usado pelo analisador semântico. Ela mantém uma **pilha de escopos** (dicionários nome → símbolo; o índice 0 é o escopo global) e um **histórico** de todos os símbolos já declarados, inclusive de escopos fechados, que é exibido na aba "Semântico & Símbolos" da interface didática com nome, tipo, escopo (`Global`, `Func:nome`, `Class:nome`, `PureBlock`, `Local`) e linha.

6.1 Pseudocódigo da tabela de símbolos

```
Classe Simbolo:
    Atributos: nome, tipo, parametros (Lista), atributos (Dicionário), metodos (Dicionário)

Classe TabelaDeSimbolos implementa ITabelaSimbolos:
    Atributos:
        escopos: Pilha de Dicionários = [ {} ]       # começa com o escopo global
        historico: Lista de Registros = []

    Método entrar_escopo():  escopos.empilhar({})
    Método sair_escopo():    Se tamanho(escopos) > 1: escopos.desempilhar()

    Método declarar(nome, tipo, rotulo_do_escopo, linha, parametros):
        Se nome existir em escopos.topo(): Retornar Falso        # redeclaração
        escopos.topo()[nome] = Novo Simbolo(nome, tipo, parametros)
        historico.adicionar({nome, tipo, escopo: rotulo_do_escopo, linha, parametros})
        Retornar Verdadeiro

    Método buscar(nome):
        Para cada escopo em escopos (do topo para a base):
            Se nome existir em escopo: Retornar escopo[nome]
        Retornar Nulo

    Método profundidade(nome):          # índice do escopo onde o nome foi declarado
        Para i de tamanho(escopos) - 1 até 0:
            Se nome existir em escopos[i]: Retornar i
        Retornar Nulo

    Método obter_tabela(): Retornar cópia de historico
```

A operação `profundidade` é a base das regras de paralelismo: uma variável declarada em um escopo anterior ao início do `pure` (ou do `fork @computador`) é externa ao bloco.

7. INTERFACE DO COMPILADOR DA LINGUAGEM MINIPAR 2026.2

Conforme o enunciado, as duas interfaces de 2026.1 (terminal e Web do desenvolvedor) foram mantidas e foi acrescentada a segunda interface Web, com finalidade didática.

7.1 Interface de terminal

`python3 main.py` abre o menu (Modo Terminal ou Interface Web). No Modo Terminal, `menu.py` lista os programas de `tests/`, compila o escolhido mostrando cada fase e executa o binário. Também é possível compilar diretamente: `python3 main.py tests/teste9_mandelbrot_5nucleos.minipar`.

```
=== Compilador MiniPar ===

=== Compilando: tests/teste9_mandelbrot_5nucleos.minipar ===
[Fase 1] Analise Lexica e Sintatica...
[Fase 2] Analise Semantica e Tabela de Simbolos...
[Fase 3] Geracao de Codigo Intermediario (TAC - 3 enderecos)...
[TAC] Arquivo gerado: output/teste9_mandelbrot_5nucleos.tac
[Fase 4] Transpilacao e Chamada do GCC (-O3)...
[CODEGEN] Compilando via g++ com -std=c++20 e -O3...
[SUCESSO] Executavel: output/teste9_mandelbrot_5nucleos.exe
=== Processo Finalizado ===
```

7.2 Primeira interface Web (desenvolvedor)

Mantida de 2026.1 na rota `/`: lista de testes, editor de código, abas com o C++ gerado e o TAC e terminal de saída. Recebeu o botão de alternância para o Modo Didático e o tema claro/escuro.

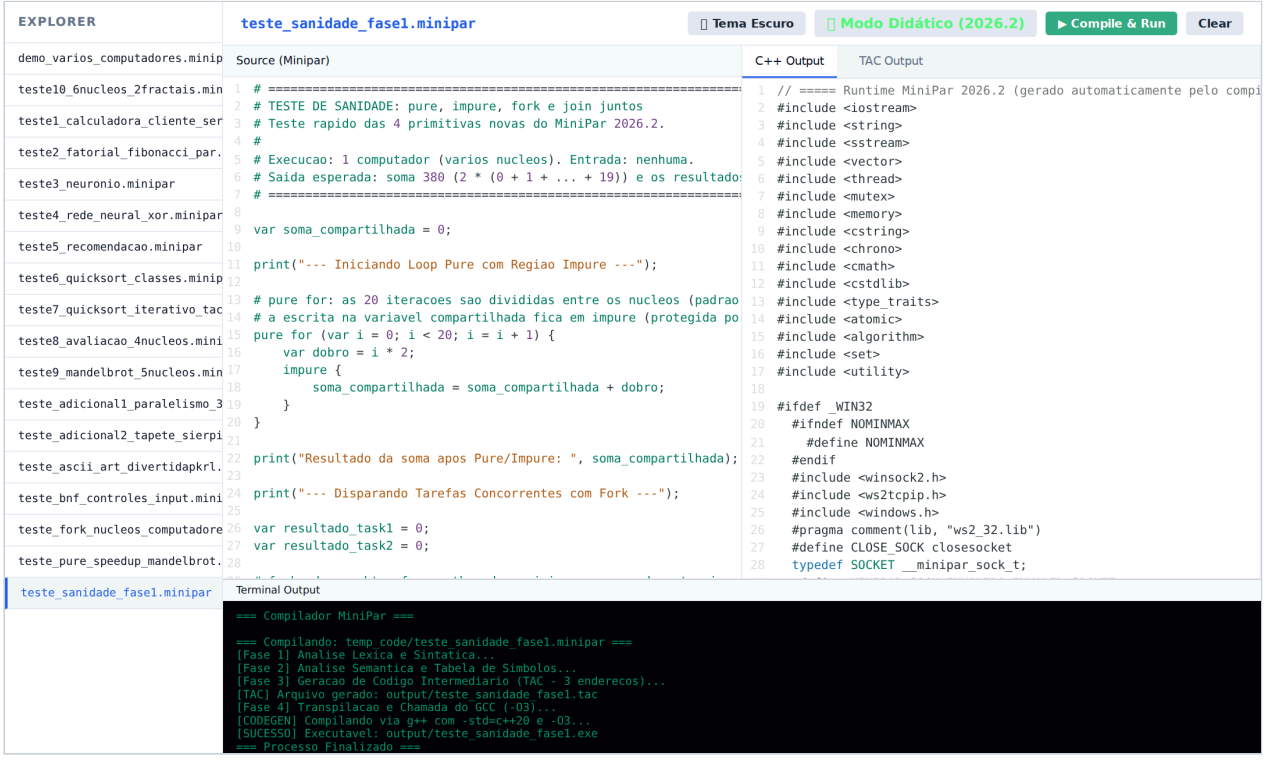


Figura 1 — Primeira interface Web (IDE do desenvolvedor) após compilar e executar o teste de sanidade.

7.3 Segunda interface Web (didática)

Na rota `/didatico`, com visual limpo inspirado no compilador educacional do IFSC. Oferece o editor com seletor de programas de teste e as opções pedidas no enunciado:

Opção do enunciado	Na interface
Tokens – Lexer	Aba **1. Léxico (Tokens)**: tabela com lexema, tipo, categoria (em cores), linha e coluna; filtro e busca.
AST – Análise Sintática	Aba **2. Árvore AST Visual**: árvore gráfica em SVG (D3.js) com zoom, arrastar, ajuste à tela, busca de nós, subárvores recolhíveis e cores para `PureBlock`, `ImpureBlock`, `ForkBlock` e `JoinStmt`.
Análise Semântica	Aba **3. Semântico & Símbolos**: Tabela de Símbolos com escopos e lista de verificações/erros.
TAC – código de 3 endereços	Aba **4. Código TAC**.
Código C++ gerado	Aba **5. C++ Transpilado**.
Compilar e baixar o executável	Botão **Baixar .exe**: compila o código que está no editor e baixa o executável.
Executar	Botão **Executar**: compila, executa e mostra a saída no terminal; a aba **6. Benchmarks** mostra speedup e tempos.

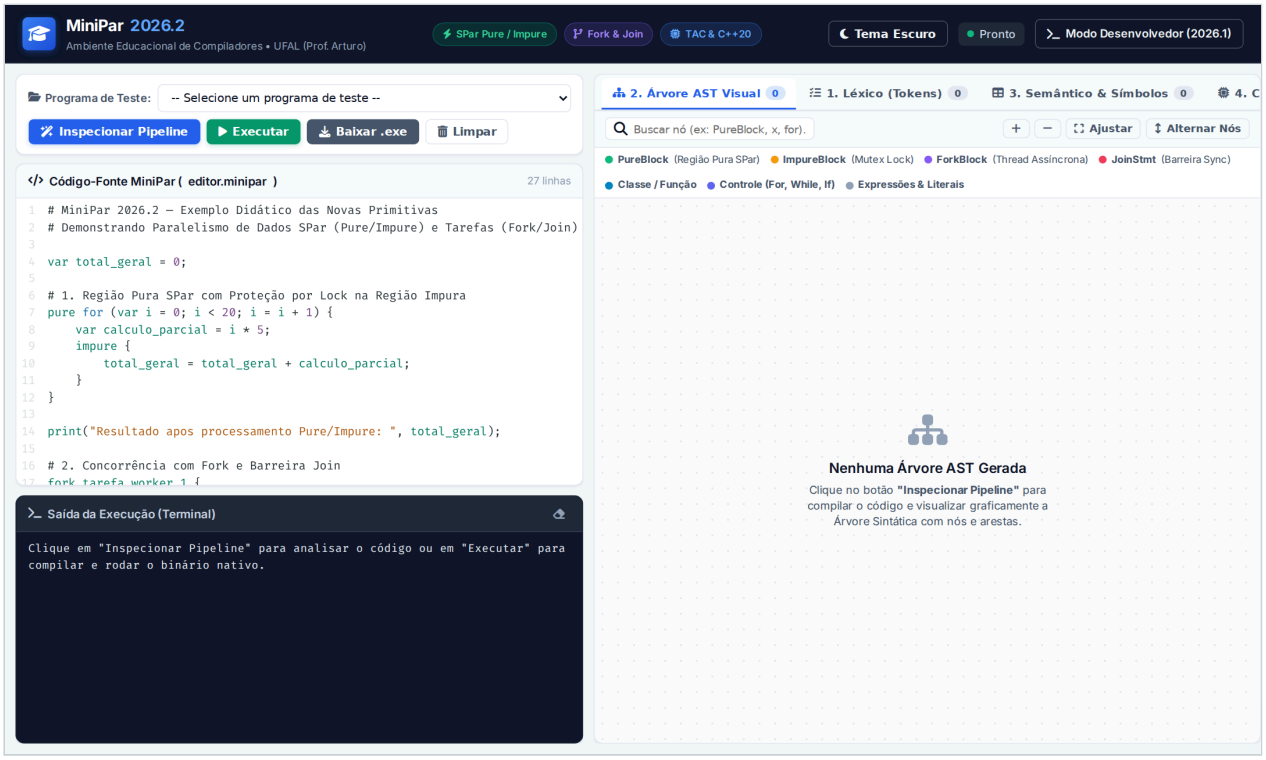


Figura 2 — Tela inicial da interface didática, com o exemplo das novas primitivas.

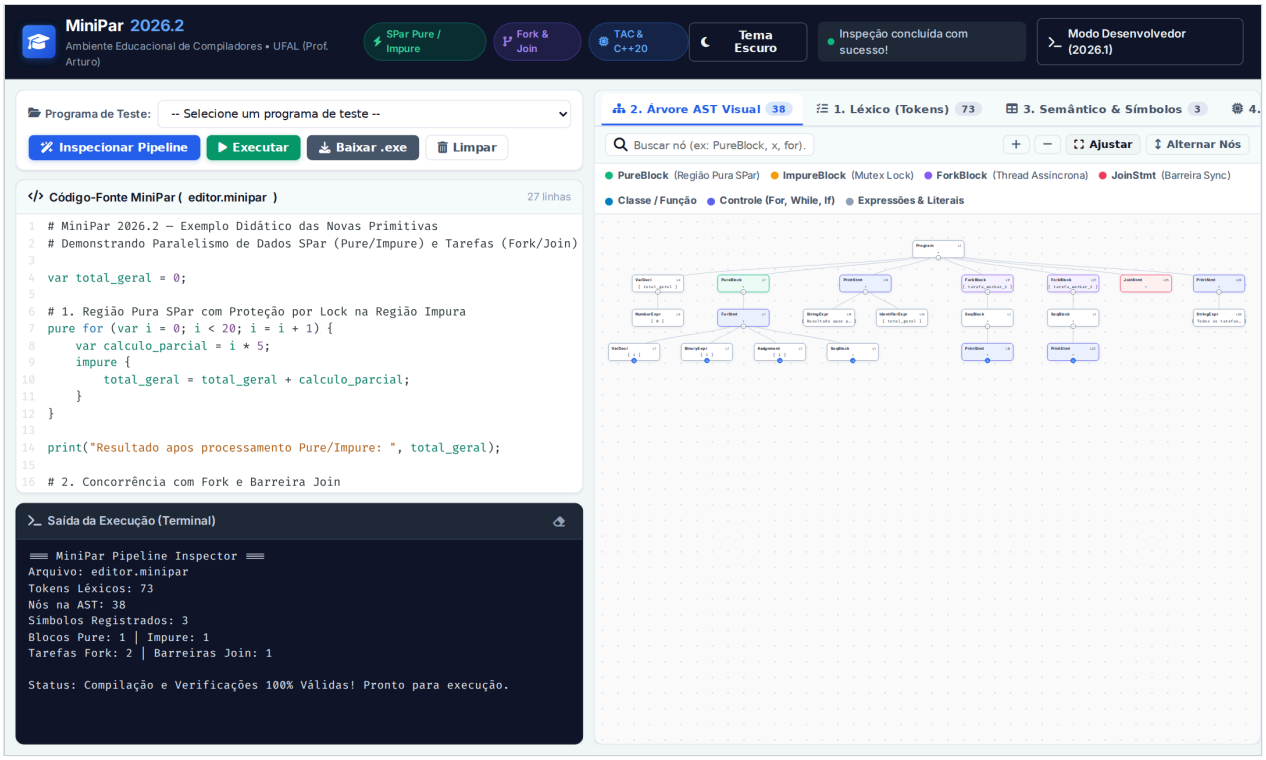


Figura 3 — Árvore AST gráfica do exemplo didático: nós PureBlock, ImpureBlock, ForkBlock e JoinStmt destacados por cor.

Os endpoints do servidor Flask são:

Endpoint	Método	Função
`/` e `/didatico`	GET	Páginas das duas interfaces Web.
`/api/tests`, `/api/get_test`, `/api/test_spec`	GET/POST	Lista os testes, devolve o código e as entradas que o programa pede.
`/api/inspect`	POST	Executa léxico, sintático, semântico, TAC e C++ e devolve tudo em JSON (tokens, AST, símbolos, erros, TAC, C++).
`/api/compile`	POST	Compila, executa com as entradas informadas e devolve saída, C++, TAC e métricas de benchmark.
`/api/build` e `/api/download_exe`	POST/GET	Compila o código do editor e baixa o `.exe`.

8. PRODUCT BACKLOG DO COMPILADOR MINIPAR 2026.2

Item	Descrição	Prioridade	Status
PB01	Estender o léxico com `pure`, `impure`, `fork`, `join` e as anotações `@pure`, `@impure`.	Alta	Concluído
PB02	Estender o parser e a AST com `PureBlock`, `ImpureBlock`, `ForkBlock` e `JoinStmt`.	Alta	Concluído
PB03	Regra semântica SPar: `impure` somente dentro de `pure`; exportar a Tabela de Símbolos com escopos.	Alta	Concluído
PB04	Gerar TAC com as diretivas de paralelismo (`BEGIN_PURE_MAP`, `ACQUIRE_LOCK`, `FORK_TASK`, `JOIN_TASKS`).	Média	Concluído
PB05	Gerar C++20 com `std::thread` e `std::mutex` para as novas primitivas.	Alta	Concluído
PB06	Implementar os Testes 8, 9 (Mandelbrot) e 10 a partir dos programas Python do enunciado e medir o speedup.	Alta	Concluído
PB07	Segunda interface Web didática na rota `/didatico`, mantendo as interfaces anteriores.	Alta	Concluído
PB08	API de inspeção (`/api/inspect`) com tokens, AST, símbolos, TAC e C++.	Alta	Concluído
PB09	Árvore AST gráfica interativa (SVG/D3.js), clara e não poluída.	Alta	Concluído
PB10	Compilar e baixar o executável pela interface.	Média	Concluído
PB11	Tema claro/escuro e alternância entre as duas interfaces Web.	Baixa	Concluído
PB12	Atualizar a gramática BNF com as novas produções.	Alta	Concluído
PB13	Modelagem UML com Componentes de Software (interfaces `I...`), casos de uso, classes e sequência; interfaces no código.	Alta	Concluído
PB14	`pure for` paralelo de fato (padrão Map) e regra contra condição de corrida em `pure`.	Alta	Concluído
PB15	Alocação da subtarefa do `fork` em núcleo (`@nucleo`) ou em outro computador (`@computador`).	Alta	Concluído
PB16	Testes automatizados de integração dos componentes e dos programas de teste.	Alta	Concluído
PB17	Relatório técnico final.	Alta	Concluído

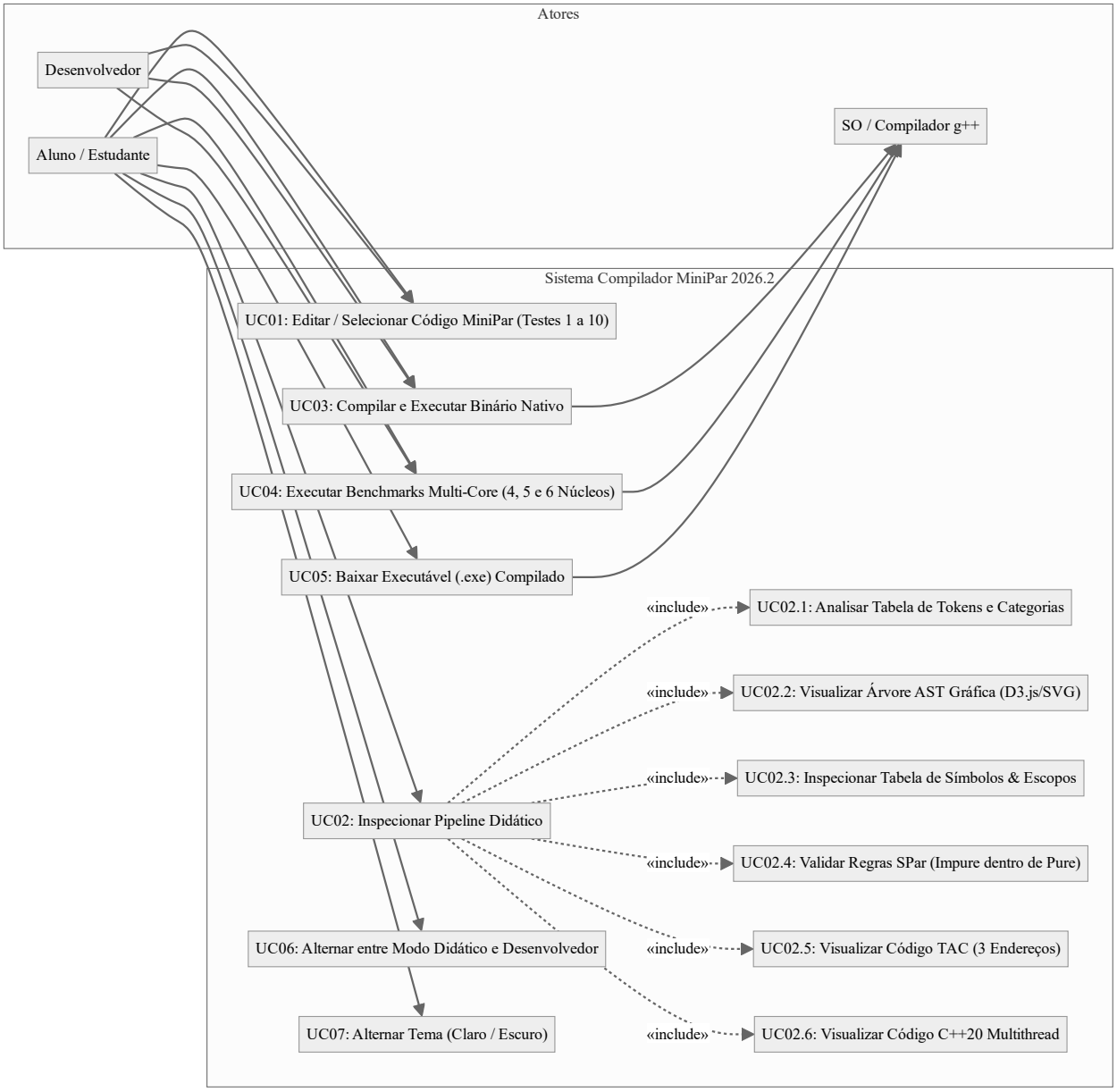
9. SPRINTS BACKLOG DO COMPILADOR MINIPAR 2026.2

Sprint	Período	Tarefa	Descrição técnica	Ref. PB
1	22/10 a 27/10	1.1	Novos tokens e palavras reservadas no `lexer.py`; anotações com `@`.	PB01
1	22/10 a 27/10	1.2	Novos nós da AST e métodos do parser para `pure`, `impure`, `fork` e `join`.	PB02
1	22/10 a 27/10	1.3	Validação de `impure` dentro de `pure`; histórico de símbolos com escopos.	PB03
1	22/10 a 27/10	1.4	Conversão da AST para JSON (`ast_to_dict`) para o visualizador.	PB02, PB09
2	28/10 a 02/11	2.1	Diretivas de paralelismo no TAC.	PB04
2	28/10 a 02/11	2.2	Runtime C++ (mutex global, pool de threads do fork, `obter_tempo`) e tradução das novas primitivas.	PB05
2	28/10 a 02/11	2.3	Primeira versão dos Testes 8, 9 e 10 e do teste de sanidade.	PB06
3	03/11 a 07/11	3.1	Rotas `/didatico`, `/api/inspect` e `/api/download_exe`.	PB07, PB08, PB10
3	03/11 a 07/11	3.2	Tela didática: tokens, símbolos, TAC, C++, benchmarks e terminal.	PB07
3	03/11 a 07/11	3.3	Visualizador gráfico da AST em D3.js (zoom, busca, recolher nós, cores).	PB09
3	03/11 a 07/11	3.4	Temas claro/escuro e alternância entre as interfaces.	PB11
4	08/11 a 11/11	4.1	BNF atualizada, diagramas UML e documentação ágil.	PB12, PB13
4	08/11 a 11/11	4.2	Revisão geral contra o enunciado: `pure for` paralelo (Map), regra de condição de corrida, `print` seguro entre threads.	PB14
4	08/11 a 11/11	4.3	`fork @nucleo` e `fork @computador` (gramática, semântico, TAC, C++, protocolo de subtarefas remotas); `c_channel` sem perda de mensagens.	PB15
4	08/11 a 11/11	4.4	Testes 8, 9 e 10 reescritos como tradução fiel dos programas Python; Teste 1 em 1 ou 2 computadores; inteiros de 64 bits.	PB06, PB15
4	08/11 a 11/11	4.5	Interfaces dos componentes no código, comentários explicativos e testes de integração automatizados.	PB13, PB16
4	08/11 a 11/11	4.6	Relatório final com telas, saídas dos testes e benchmarks.	PB17

Distribuição de responsabilidades: **Ludson Lira de Almeida** — núcleo do compilador e benchmarks; **Davi Silva de Melo Lins** — segunda interface Web e árvore AST visual; **Iury Kauann David Nogueira** — engenharia de software (BNF, UML, Scrum), revisão final e relatório.

10. DIAGRAMA DE CASOS DE USO (UML)

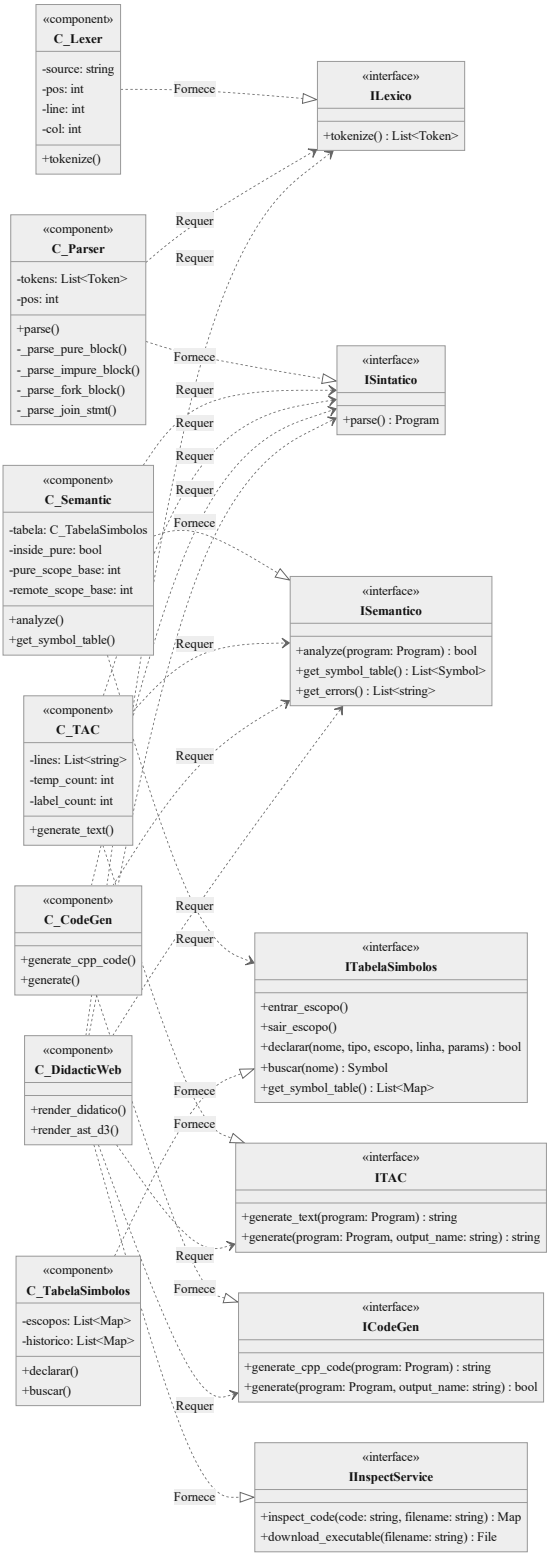
Os atores são o aluno (interface didática), o desenvolvedor (terminal e IDE) e o sistema operacional com o compilador `g++`.



11. ARQUITETURA UTILIZANDO COMPONENTES DE SOFTWARE (UML)

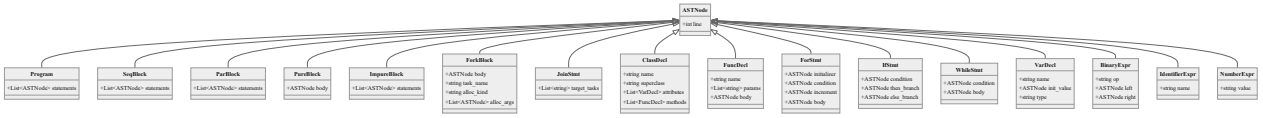
Cada fase do compilador é um componente de software com **interface fornecida** e **interfaces requeridas**. As interfaces estão implementadas como classes abstratas em `src/componentes.py` — mesmo padrão do exemplo `IRelogio`/`Relogio` do Anexo 2 do enunciado — e cada componente é a classe concreta que as implementa. O Anexo 1 pedia o acréscimo dos geradores de código: aqui estão o `C_TAC` e o `C_CodeGen`.

Componente	Classe no código	Interface fornecida	Interfaces requeridas	Arquivo
`C_Lexer`	`Lexer`	`ILexico`	—	`src/lexer.py`
`C_Parser`	`Parser`	`ISintatico`	`ILexico`	`src/parser.py`
`C_Semantic`	`SemanticAnalyzer`	`ISemantico`	`ISintatico`, `ITabelaSimbolos`	`src/semantic.py`
`C_TabelaSimbolos`	`TabelaSimbolos`	`ITabelaSimbolos`	—	`src/semantic.py`
`C_TAC`	`TACGenerator`	`ITAC`	`ISintatico`, `ISemantico`	`src/tac.py`
`C_CodeGen`	`CppCodeGenerator`	`ICodeGen`	`ISintatico`, `ISemantico`	`src/codegen.py`
`C_DidacticWeb`	rotas Flask + `didatico.js`	`IInspectService`	todas as anteriores	`app.py`


«component»
C_Lexer
-source: string
-pos: int
-line: int
-col: int
+tokenize()
Fornece
«interface»
ILexico
+tokenize() : List<Token>
«component»
C_Parser
-tokens: List<Token>
-pos: int
+parse()
-_parse_pure_block()
-_parse_impure_block()
-_parse_fork_block()
-_parse_join_stmt()
Requer
Requer
Fornece
«interface»
ISintatico
+parse() : Program
Requer
Requer
Requer
Requer
«component»
C_Semantic
-tabela: C_TabelaSimbolos
-inside_pure: bool
-pure_scope_base: int
-remote_scope_base: int
+analyze()
+get_symbol_table()
Fornece
«interface»
ISemantico
+analyze(program: Program) : bool
+get_symbol_table() : List<Symbol>
+get_errors() : List<string>
Requer
«component»
C_TAC
-lines: List<string>
-temp_count: int
-label_count: int
+generate_text()
Requer
«component»
C_CodeGen
+generate_cpp_code()
+generate()
Requer
Requer
«interface»
ITabelaSimbolos
+entrar_escopo()
+sair_escopo()
+declarar(nome, tipo, escopo, linha, params) : bool
+buscar(nome) : Symbol
+get_symbol_table() : List<Map>
«component»
C_DidacticWeb
+render_didatico()
+render_ast_d3()
Fornece
Fornece
«interface»
ITAC
+generate_text(program: Program) : string
+generate(program: Program, output_name: string) : string
Requer
«component»
C_TabelaSimbolos
-escopos: List<Map>
-historico: List<Map>
+declarar()
+buscar()
Fornece
«interface»
ICodeGen
+generate_cpp_code(program: Program) : string
+generate(program: Program, output_name: string) : bool
Requer
«interface»
IInspectService
+inspect_code(code: string, filename: string) : Map
+download_executable(filename: string) : File
Fornece

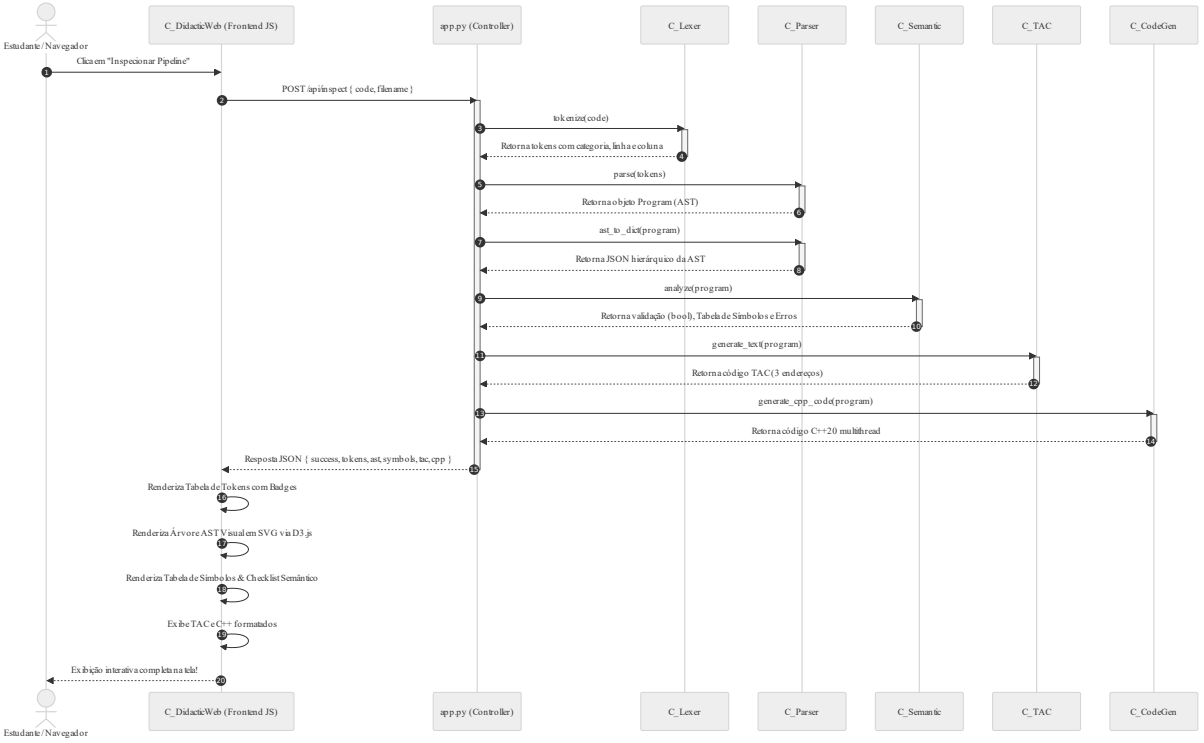
12. MODELAGEM DOS COMPONENTES VIA DIAGRAMA DE CLASSES (UML)

12.1 Classes dos nós da AST

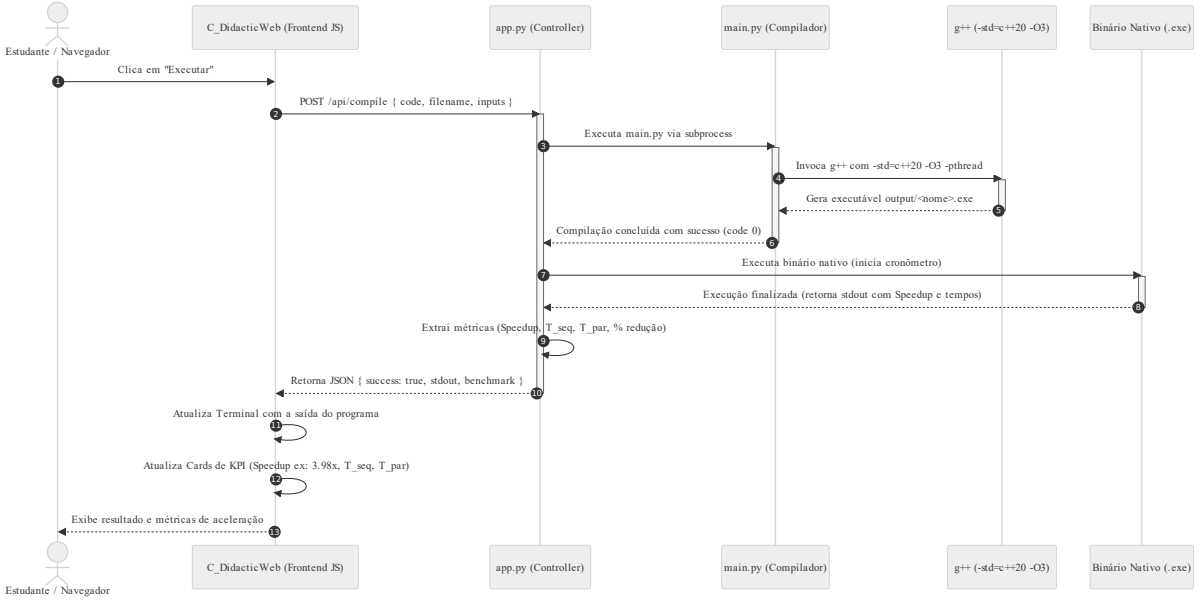


12.2 Diagramas de sequência

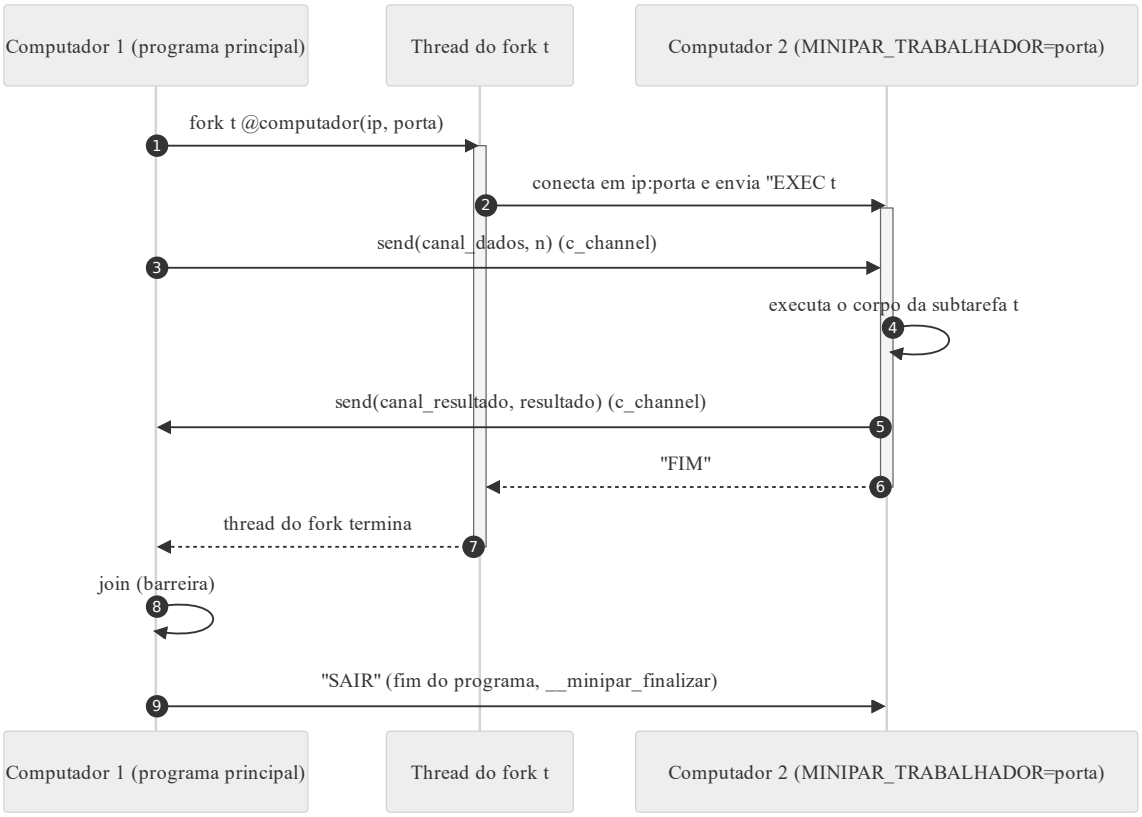
Inspeção do pipeline pela interface didática (`POST /api/inspect`):



Compilação, execução e benchmark (`POST /api/compile`):



Subtarefa `fork` alocada em outro computador:



13. TESTES DE SOFTWARE DE INTEGRAÇÃO DOS COMPONENTES

A integração é verificada por uma suíte automatizada (`tests/test_integracao_componentes.py`, 33 testes), executada com `python3 -m unittest discover -s tests -v`. Ela vai do componente isolado ao sistema completo:

Grupo	O que integra	Exemplos de verificação
T1	Componentes × interfaces	Cada classe é instância da sua interface (`ILexico`, `ISintatico`, ...).
T2	`C_Lexer`	Palavras reservadas e anotações; comentários descartados; escapes de string; linha e coluna.
T3	`C_Lexer` → `C_Parser`	`pure for` com `impure` vira `PureBlock`/`ImpureBlock`; `fork` com `@nucleo` e `@computador`; precedência; `ParseError` com a linha.
T4	`C_Parser` → `C_Semantic` / `C_TabelaSimbolos`	`impure` fora de `pure`; condição de corrida em `pure`; `fork @computador` com variável externa; erros clássicos; escopos na tabela.
T5	`C_Semantic` → `C_TAC` / `C_CodeGen`	Diretivas de paralelismo no TAC; TAC do Teste 7 gerado pelo compilador; C++ com `__minipar_pure_map`, mutex e alocação do fork.
T6	Todos os componentes + `g++` + execução	Os 18 programas compilam e produzem a saída esperada; o Mandelbrot é idêntico ao do Python do professor; `pure` dá o mesmo resultado com 1 ou 4 threads; Teste 1 estável.
T7	`C_DidacticWeb` + componentes	`/api/inspect` devolve todas as fases; erros sintático e semântico aparecem; métricas de benchmark; compilar e baixar o executável.

Saída da execução da suíte:

```
test_cada_componente_implementa_sua_interface ... ok
test_comentarios_sao_descartados ... ok
test_linha_e_coluna ... ok
test_nucleo_sem_arroba_continua_identificador ... ok
test_palavras_reservadas_e_anotacoes ... ok
test_string_com_escapes_e_operadores_duplos ... ok
test_erro_sintatico_lanca_parse_error_com_linha ... ok
test_fork_com_alocacao_em_nucleo_e_computador ... ok
test_fork_de_chamada_sem_nome_nao_e_ambiguo ... ok
test_precedencia_dos_operadores ... ok
test_pure_for_com_impure_vira_nos_da_ast ... ok
test_condicao_de_corrida_em_pure ... ok
test_erros_classicos ... ok
test_escrita_em_posicao_propria_dentro_de_pure_e_permitida ... ok
test_fork_remoto_nao_acessa_variavel_externa ... ok
test_impure_fora_de_pure ... ok
test_programa_valido ... ok
test_tabela_de_simbolos_registra_escopos ... ok
test_cpp_traduz_pure_impure_e_fork_com_alocacao ... ok
test_tac_do_teste7_e_gerado_pelo_compilador ... ok
test_tac_tem_as_diretivas_de_paralelismo ... ok
test_entrada_invalida_nao_trava_o_teste1
Sem entradas (caixa vazia na web), to_number recebe "": avisa e usa 0, sem travar. ... ok
test_mandelbrot_igual_ao_do_programa_python_do_professor
As 20 linhas do fractal (testes 9 e 10) são idênticas às do Python do enunciado. ... ok
test_pure_da_o_mesmo_resultado_com_1_ou_varias_threads ... ok
test_saidas_esperadas ... ok
test_teste1_sockets_estavel ... ok
test_teste5_recomenda_os_12_produtos_do_enunciado ... ok
test_todos_os_programas_compilam ... ok
test_compilar_e_baixar_executavel ... ok
test_inspect_mostra_erro_sintatico_e_semantico ... [ERRO SINTATICO] Linha 1: Expressao esperada
ok
test_inspect_retorna_todas_as_fases ... ok
test_metricas_de_benchmark ... ok
test_nome_de_arquivo_nao_sai_da_pasta_tests ... ok

----------------------------------------------------------------------
Ran 33 tests in 54.995s
```

```
OK
```

14. TESTES DOS PROGRAMAS DE EXEMPLO NA LINGUAGEM MINIPAR 2026.2

Todos os programas estão em `tests/`, comentados com `#` e com um cabeçalho que informa o que o programa faz, as entradas e se a execução é em um computador ou em vários. As saídas abaixo foram capturadas das execuções reais (Linux/WSL2, g++ 15.2 `-O3`).

14.1 Teste 1 — Calculadora cliente/servidor (`c_channel`)

O cliente lê a operação e os operandos e os envia ao servidor, que calcula e devolve o resultado. O servidor é uma subtarefa `fork servidor @computador(...)`: sem computador extra o programa avisa e roda o servidor em um processo separado no mesmo computador (teste em `localhost`); informando o IP, o servidor roda em um segundo computador, como pede o enunciado. Entradas: `*`, `7`, `6`.

```
==== Calculadora cliente-servidor ====
Opcoes: +, -, *, /
Operacao (+,-,*,/): Operando 1: Operando 2: IP do computador servidor [ENTER = este computador]:
[fork servidor] Nenhum computador extra aberto em 127.0.0.1:5100 -> a subtarefa vai rodar NESTE
computador, em um processo separado.
[computador :5100] trabalhador pronto, aguardando subtarefas...
[fork servidor] subtarefa enviada ao computador 127.0.0.1:5100
[computador :5100] executando subtarefa servidor
Servidor recebeu: * 7 6
Resultado: 42
[fork servidor] subtarefa concluida no computador 127.0.0.1:5100
```

14.2 Teste 2 — Fatorial e Fibonacci em paralelo (`par`)

```
Fibonacci:
0
1
1
2
3
5
8
13
Fatorial de 5: 120
```

14.3 Teste 3 — Neurônio (classes e objetos)

O neurônio aprende na iteração 51, como na execução do enunciado.

```
==== Teste 3: neuronio simples com classes e objetos ====
Entrada: 1 Saida desejada: 0
Iteracao: 1
Soma ponderada: 1
Saida: 1
Erro: -1
Peso entrada: 0.5
Peso bias: 0.5
Iteracao: 2
Soma ponderada: 0.98
Saida: 1
Erro: -1
Peso entrada: 0.49
Peso bias: 0.49
... (iteracoes 3 a 49 omitidas) ...
Peso bias: 0.02
Iteracao: 50
Soma ponderada: 0.02
Saida: 1
```

```
Erro: -1
Peso entrada: 0.01
Peso bias: 0.01
Iteracao: 51
Soma ponderada: -6.17562e-16
Saida: 0
Erro: 0
Peso entrada: -3.08781e-16
Peso bias: -3.08781e-16
Neuronio aprendeu dinamicamente.
Iteracoes totais: 51
Peso final: -3.08781e-16
Peso bias final: -3.08781e-16
Valor desejado: 0
```

14.4 Teste 4 — Rede neural XOR

```
==== Teste 4: rede neural XOR com backpropagation ====
Input: [0, 0], Predicted Output: 0.00684395
Input: [0, 1], Predicted Output: 0.986493
Input: [1, 0], Predicted Output: 0.98925
Input: [1, 1], Predicted Output: 0.0140296
```

14.5 Teste 5 — Sistema de recomendação

Com o histórico do enunciado (Smartphone, Jeans, Micro-ondas e Ficção), o programa recomenda os mesmos 12 produtos da execução em Python.

```
==== Teste 5: recomendacao e-commerce com rede neural ====
Informe o historico de compras (1 = comprou, 0 = nao comprou):
Smartphone - comprou? (0/1):
Laptop - comprou? (0/1):
Tablet - comprou? (0/1):
Fones de ouvido - comprou? (0/1):
Camisa - comprou? (0/1):
Jeans - comprou? (0/1):
Jaqueta - comprou? (0/1):
Sapatos - comprou? (0/1):
Geladeira - comprou? (0/1):
Micro-ondas - comprou? (0/1):
Maquina de lavar - comprou? (0/1):
Ar condicionado - comprou? (0/1):
Ficcao - comprou? (0/1):
Nao-ficcao - comprou? (0/1):
Ficcao cientifica - comprou? (0/1):
Fantasia - comprou? (0/1):
Produtos recomendados para voce:
Laptop probabilidade: 0.723522
Tablet probabilidade: 0.711771
Fones de ouvido probabilidade: 0.716263
Camisa probabilidade: 0.564145
Jaqueta probabilidade: 0.723522
Sapatos probabilidade: 0.711771
Geladeira probabilidade: 0.716263
Maquina de lavar probabilidade: 0.7191
Ar condicionado probabilidade: 0.723522
Nao-ficcao probabilidade: 0.716263
Ficcao cientifica probabilidade: 0.564145
Fantasia probabilidade: 0.7191
```

14.6 Teste 6 — Quicksort com classes

```
==== Ordenacao com Quicksort ====
Insira os elementos do vetor separados por espaco:
Vetor original: [10, -3, -40, 80, 70, -100]
Vetor ordenado: [-100, -40, -3, 10, 70, 80]
```

14.7 Teste 7 — Quicksort não recursivo e código de três endereços

```
Array ordenado: [5, 12, 33, 61, 98]
```

O TAC abaixo é o gerado pelo compilador a partir do programa MiniPar (início do arquivo `output/teste7_quicksort_iterativo_tac.tac`). O TAC de referência do enunciado, escrito para o programa Python, está em `tests/expected_results/`; os dois representam o mesmo algoritmo (pilha de intervalos, particionamento e trocas), com nomes de temporários e rótulos diferentes.

```
func quicksort_iterativo:
t0 = [0, 4, 0, 0, 0, 0, 0, 0, 0]
stack = t0
sp = 2
WHILE_0:
t1 = sp > 0
ifFalse t1 goto ENDWHILE_1
t2 = sp - 1
sp = t2
t3 = stack[sp]
fim = t3
t4 = sp - 1
sp = t4
t5 = stack[sp]
inicio = t5
t6 = inicio >= fim
ifFalse t6 goto ENDIF_2
goto WHILE_0
ENDIF_2:
t7 = arr[fim]
pivot = t7
t8 = inicio - 1
i = t8
j = inicio
WHILE_3:
t9 = j < fim
ifFalse t9 goto ENDWHILE_4
t10 = arr[j]
t11 = t10 <= pivot
ifFalse t11 goto ENDIF_5
t12 = i + 1
i = t12
t13 = arr[i]
tmp = t13
...
```

14.8 Testes 8, 9 e 10 — algoritmos paralelos com `fork` e `join`

Os três são **traduções fiéis dos programas Python do enunciado** (cópias em `docs/referencia_python/`): mesmo gerador pseudoaleatório (LCG, mesma semente e constantes — a sequência gerada é idêntica à do Python), mesma carga de CPU por atividade (`int(segundos × 1.500.000)` iterações de soma) e o mesmo fractal. Os processos do Python viram subtarefas `fork ... @nucleo(n)`, e no `join` cada programa confere que o modo paralelo realizou exatamente o mesmo trabalho do sequencial.

Teste 8 — avaliação de 160 atividades em 4 núcleos (Professor + 3 Monitores):

```
Total de Atividades a Avaliar: 160 (40 alunos x 4 atividades)

--- 1. Executando Modo Sequencial (Apenas 1 Nucleo / Professor) ---
Tempo Sequencial Concluido em: 0.0142522 segundos

--- 2. Executando Modo Paralelo (4 Nucleos Dedicados: Professor + 3 Monitores) ---
Tempo Paralelo Concluido em:   0.00583196 segundos

Verificacao (join): as 160 correcoes paralelas conferem com as sequenciais (OK)
```

```
========================================
         BENCHMARK DE DESEMPENHO
========================================
Tempo Sequencial : 0.0142522 s
Tempo Paralelo   : 0.00583196 s
Aceleracao       : 2.44381 x mais rapido!
Reducao no tempo : 59.0803 % de economia
========================================
```

Teste 9 — 5 núcleos (4 avaliadores + Fractal de Mandelbrot). As 20 linhas do fractal são idênticas, caractere por caractere, às do programa Python. As linhas são calculadas com `pure for` e o total de iterações é acumulado em `impure`.

```
Carga de Trabalho: 160 atividades + 1 Calculo de Fractal ASCII

--- 1. Executando Modo Sequencial (Apenas 1 Nucleo) ---
Tempo Sequencial Concluido em: 0.014107 segundos

--- 2. Executando Modo Paralelo (5 Nucleos Dedicados) ---
Tempo Paralelo Concluido em:   0.00552106 segundos


=== FRACTAL DE MANDELBROT (Gerado pelo Nucleo 5) ===


                                .
                              ..=:.
                               @@@:
                           +@.@@@@@@@@@@. :..
                          .@@@@@@@@@@@@@@:
                 .      #@@@@@@@@@@@@@@@@@@:
             .@:@@@- :@@@@@@@@@@@@@@@@@@@@:
          - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@.
          - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@
             .@:@@@- :@@@@@@@@@@@@@@@@@@@@:
                .      #@@@@@@@@@@@@@@@@@@:
                         .@@@@@@@@@@@@@@@@:
                           +@.@@@@@@@@@@. :..
                                 @@@:
                                ..=:.
                                   .

==================================================
Iteracoes calculadas (acumuladas na regiao impure): 26553

Verificacao (join): avaliacoes e fractal do modo paralelo conferem com o sequencial (OK)
========================================
         BENCHMARK DE DESEMPENHO
========================================
Nucleos Utilizados (Paralelo) : 5
Tempo Sequencial              : 0.014107 s
Tempo Paralelo                : 0.00552106 s
Aceleracao (Speedup)          : 2.55512 x mais rapido!
Reducao de Tempo              : 60.863 %
========================================
```

Teste 10 — 6 núcleos (avaliadores + Mandelbrot + Tapete de Sierpinski). No Python o Sierpinski é desenhado com Turtle (janela gráfica); no MiniPar é desenhado em ASCII pelo 6º núcleo.

```
Carga de Trabalho: 160 atividades + Mandelbrot (ASCII) + Sierpinski (ASCII)

--- 1. Executando Modo Sequencial (Apenas 1 Nucleo) ---
Tempo Sequencial Concluido em: 0.0130098 segundos

--- 2. Executando Modo Paralelo (6 Nucleos Dedicados) ---
=== TAPETE DE SIERPINSKI (Gerado pelo Nucleo 6) ===
################################################
##  ####  ####  ####  ####  ####  ####  ####  ##
```

```
######################################################
######      ############      ############      ######
##  ##      ##  ####  ##      ##  ####  ##      ##  ##
######      ############      ############      ######
######################################################
##  ####  ####  ####  ####  ####  ####  ####  ####  ##
######################################################
##################                  ##################
##  ####  ####  ##                    ##  ####  ####  ##
##################                  ##################
######      ######                    ######      ######
##  ##      ##  ##                    ##  ##      ##  ##
######      ######                    ######      ######
##################                  ##################
##  ####  ####  ##                    ##  ####  ####  ##
##################                  ##################
######################################################
##  ####  ####  ####  ####  ####  ####  ####  ####  ##
######################################################
######      ############      ############      ######
##  ##      ##  ####  ##      ##  ####  ##      ##  ##
######      ############      ############      ######
######################################################
##  ####  ####  ####  ####  ####  ####  ####  ####  ##
######################################################
==================================================
Celulas preenchidas (acumuladas na regiao impure): 512
-> [Nucleo 6] Tapete de Sierpinski renderizado com sucesso na tela!
Tempo Paralelo Concluido em:   0.00357437 segundos


=== FRACTAL DE MANDELBROT (Gerado pelo Nucleo 5) ===


                                .
                              ..=:.
                               @@@:
                          +@.@@@@@@@@@@. :..
                         .@@@@@@@@@@@@@@:
                 .     #@@@@@@@@@@@@@@@@@@:
              .@:@@@- :@@@@@@@@@@@@@@@@@@@:
           - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@
@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@@.
           - .@@@@@@@@@@@@@@@@@@@@@@@@@@@@
              .@:@@@- :@@@@@@@@@@@@@@@@@@@:
                 .     #@@@@@@@@@@@@@@@@@@:
                        .@@@@@@@@@@@@@@@:
                          +@.@@@@@@@@@@. :..
                               @@@:
                              ..=:.
                                 .

==================================================
Iteracoes calculadas (acumuladas na regiao impure): 26553

Verificacao (join): avaliacoes e Mandelbrot do modo paralelo conferem com o sequencial (OK)
==========================================
         BENCHMARK DE DESEMPENHO
==========================================
Nucleos Utilizados (Paralelo) : 6
Tempo Sequencial              : 0.0130098 s
Tempo Paralelo                : 0.00357437 s
Aceleracao (Speedup)          : 3.63974 x mais rapido!
Reducao de Tempo              : 72.5255 %
==========================================
```

14.8.1 Benchmarks

Medições na mesma máquina (CPU com 16 núcleos lógicos; mediana de 5 execuções):

Teste	Núcleos	MiniPar (C++): sequencial → paralelo	Speedup MiniPar	Python do enunciado: sequencial → paralelo	Speedup Python
Teste 8	4	12,1 ms → 3,8 ms	~3,3x	370 ms → 140 ms	2,6x
Teste 9	5	13,1 ms → 3,5 ms	~3,9x	410 ms → 140 ms	2,8x
Teste 10	6	13,0 ms → 3,4 ms	~3,8x	não executado (requer janela gráfica do Turtle)	—
Pure × Sequencial	16 threads	1,86 s → 0,18 s	~10x	—	—

O mesmo trabalho é executado cerca de 30 vezes mais rápido pelo código compilado do que pelo Python interpretado. Como os tempos do MiniPar nos Testes 8 a 10 são de poucos milissegundos, o speedup varia entre execuções (aproximadamente de 3,1x a 4,0x); o limite teórico é 4x, pois as 160 atividades são divididas entre 4 núcleos e os fractais levam menos de 1 ms. No enunciado, o professor registrou 1,18x, 1,02x e 0,52x com o Python em outra máquina.

14.9 Testes adicionais

Pure × Sequencial (speedup do paralelismo de dados). O mesmo Mandelbrot pesado (1600×1200, até 2000 iterações por ponto) é calculado com `for` comum e com `pure for`; o programa confere que os resultados são iguais.

```
Carga: Mandelbrot 1600 x 1200 com ate 2000 iteracoes por ponto

--- 1. Executando Modo Sequencial (laco for comum, 1 nucleo) ---
Pontos dentro do conjunto: 402553
Tempo Sequencial Concluido em: 1.86749 segundos

--- 2. Executando Modo Paralelo (pure for: iteracoes divididas entre os nucleos) ---
Pontos dentro do conjunto: 402553
Tempo Paralelo Concluido em: 0.1675 segundos

========================================
BENCHMARK PURE x SEQUENCIAL
========================================
Verificacao                  : mesmo resultado nas duas versoes (OK)
Tempo Sequencial             : 1.86749 s
Tempo Paralelo               : 0.1675 s
Aceleracao (Speedup)         : 11.1492 x mais rapido!
Reducao de Tempo             : 91.0308 %
========================================
```

Escalabilidade variando o número de threads do `pure` (`MINIPAR_PURE_THREADS`):

Threads	1	2	4	8	16
Speedup	1,00x	1,97x	3,92x	6,75x	~10–12x

Fork com alocação em núcleos e em computadores. A mesma tarefa é dividida em 2 subtarefas, primeiro em 2 núcleos e depois em 2 computadores (aqui em `localhost`, com os processos trabalhadores iniciados automaticamente):

```
=== Parte 1: fork com subtarefas alocadas em NUCLEOS da CPU ===
[subtarefa_soma] executada no nucleo 0
[subtarefa_fatorial] executada no nucleo 1
Soma de 1 a 10000 = 50005000
Fatorial de 12 = 479001600

=== Parte 2: fork com subtarefas alocadas em COMPUTADORES diferentes ===
[fork subtarefa_soma_remota] Nenhum computador extra aberto em 127.0.0.1:6201 -> a subtarefa vai
rodar NESTE computador, em um processo separado.
```

```
[fork subtarefa_fatorial_remota] Nenhum computador extra aberto em 127.0.0.1:6202 -> a subtarefa
vai rodar NESTE computador, em um processo separado.
[computador :6201] trabalhador pronto, aguardando subtarefas...
[computador :6202] trabalhador pronto, aguardando subtarefas...
[fork subtarefa_soma_remota] subtarefa enviada ao computador 127.0.0.1:6201
[fork subtarefa_fatorial_remota] subtarefa enviada ao computador 127.0.0.1:6202
[computador :6201] executando subtarefa subtarefa_soma_remota
[computador :6202] executando subtarefa subtarefa_fatorial_remota
[fork subtarefa_soma_remota] subtarefa concluida no computador 127.0.0.1:6201
[fork subtarefa_fatorial_remota] subtarefa concluida no computador 127.0.0.1:6202
Soma de 1 a 10000 (calculada no computador 2) = 50005000
Fatorial de 12 (calculado no computador 3) = 479001600
```

Teste de sanidade (pure, impure, fork e join):

```
--- Iniciando Loop Pure com Regiao Impure ---
Resultado da soma apos Pure/Impure:  380
--- Disparando Tarefas Concorrentes com Fork ---
Tarefa A concluida.
Pai aguardando tarefas via Join...
Tarefa B concluida.
Resultado Tarefa A:  150
Resultado Tarefa B:  500
--- Teste de Sanidade Fase 1 Finalizado com Sucesso! ---
```

Paralelismo com 3 "computadores" trocando mensagens (herdado de 2026.1):

```
==== Teste adicional 1: paralelismo real com 3 computadores ====
Menu do Computador 1
1 - Quick sort no Computador 1: 127.0.0.1:5301
2 - Multiplicacao de matrizes no Computador 2: 127.0.0.1:5302
3 - Fatorial no Computador 3: 127.0.0.1:5303
Resultados recolhidos no Computador 1:
PC1 QuickSort: [33,12,98,5,61] -> [5,12,33,61,98]
PC2 Matrizes: [[1,2],[3,4]] x [[5,6],[7,8]] -> [[19,22],[43,50]]
PC3 Fatorial: 5! = 120
```

Tapete de Sierpinski recursivo e controles da gramática:

```
==== Teste adicional 2: tapete de Sierpinski ====
[#, #, #, #, #, #, #, #, #]
[#,  , #, #,  , #, #,  , #]
[#, #, #, #, #, #, #, #, #]
[#, #, #,  ,  ,  , #, #, #]
[#,  , #,  ,  ,  , #,  , #]
[#, #, #,  ,  ,  , #, #, #]
[#, #, #, #, #, #, #, #, #]
[#,  , #, #,  , #, #,  , #]
[#, #, #, #, #, #, #, #, #]
```

```
Controle OK: 7 2
```

14.10 Tabela de estados dos testes

Teste	Cenário coberto	Resultado observado
teste1_calculadora_cliente_servidor	`c_channel`, `send`/`receive`, `fork @computador` (1 ou 2 computadores)	Servidor recebeu `* 7 6`; `Resultado: 42`.
teste2_fatorial_fibonacci_par	Bloco `par` com duas threads	Fatorial de 5 = 120; Fibonacci até 13.
teste3_neuronio	Classe, métodos, `this`, `while`	Convergiu em 51 iterações.
teste4_rede_neural_xor	Funções, classe, backpropagation, `exp`	≈0 para [0,0] e [1,1]; ≈1 para [0,1] e [1,0].

Teste	Cenário coberto	Resultado observado
teste5_recomendacao	Listas, classes, rede neural	12 produtos recomendados (iguais aos do enunciado).
teste6_quicksort_classes	OO, recursão, listas, `input`	`[-100, -40, -3, 10, 70, 80]`.
teste7_quicksort_iterativo_tac	Funções, pilha manual, TAC	`[5, 12, 33, 61, 98]`.
teste8_avaliacao_4nucleos	`fork @nucleo` (4 núcleos), `join`	Paralelo confere com sequencial; speedup ~3,3x.
teste9_mandelbrot_5nucleos	`fork` (5 núcleos), `pure`/`impure`, Mandelbrot	Fractal idêntico ao do Python; speedup ~3,9x.
teste10_6nucleos_2fractais	`fork` (6 núcleos), 2 fractais	Paralelo confere com sequencial; speedup ~3,8x.
teste_pure_speedup_mandelbrot	`pure for` com carga pesada	Mesmo resultado; speedup ~10x (16 threads).
teste_fork_nucleos_computadores	`@nucleo` e `@computador`	Mesmos resultados nas duas alocações.
demo_varios_computadores	`@computador` com IPs informados na execução	Subtarefas executadas nos outros computadores.
teste_sanidade_fase1	`pure`, `impure`, `fork`, `join`	Soma 380; resultados 150 e 500.
teste_adicional1 / adicional2 / ascii_art / bnf_controles	Canais, recursão, matrizes, controles	Saídas esperadas.

15. IMPLEMENTAÇÃO

15.a Tecnologias utilizadas

Tecnologia	Uso
Python 3 (3.10 ou superior)	Implementação do compilador (orientada a objetos), terminal, menu e servidor Web.
Flask 3.0.0	Servidor das duas interfaces Web.
CodeMirror e D3.js v7	Editor de código e desenho da árvore AST em SVG.
C++20 e g++ (`-O3 -pthread`)	Linguagem alvo e compilação para o executável `.exe`.
`std::thread`, `std::mutex`, `std::atomic`	`par`, `fork`/`join`, `pure for` (Map) e `impure`.
Afinidade de CPU (`pthread_setaffinity_np` / `SetThreadAffinityMask`)	`fork @nucleo(n)`.
Sockets TCP	`c_channel`, `send`, `receive` e o protocolo do `fork @computador`.
`unittest`	Testes automatizados de integração.
Git e GitHub	Versionamento, com *branches* por funcionalidade e *pull requests*.
Ambiente	Linux (WSL2 – Ubuntu) e VS Code.

O código do compilador contém comentários explicativos: cada arquivo de `src/` começa com a descrição do componente, e cada método do parser traz a produção BNF que implementa.

15.a.1 Geração de código intermediário (TAC)

O módulo `src/tac.py` percorre a AST e emite instruções com no máximo um operador por linha, usando temporários (`t0`, `t1`, ...) e rótulos com desvios `goto`, `if x goto L` e `ifFalse x goto L`. As novas primitivas geram diretivas próprias. TAC do teste de sanidade:

```
soma_compartilhada = 0
print "--- Iniciando Loop Pure com Regiao Impure ---"
BEGIN_PURE_MAP
i = 0
FOR_0:
t0 = i < 20
ifFalse t0 goto ENDFOR_2
t1 = i * 2
dobro = t1
ACQUIRE_LOCK
t2 = soma_compartilhada + dobro
soma_compartilhada = t2
RELEASE_LOCK
FOR_STEP_1:
t3 = i + 1
i = t3
goto FOR_0
ENDFOR_2:
END_PURE_MAP
print "Resultado da soma apos Pure/Impure: "
print soma_compartilhada
print "--- Disparando Tarefas Concorrentes com Fork ---"
resultado_task1 = 0
resultado_task2 = 0
FORK_TASK tarefa_a:
t4 = 100 + 50
resultado_task1 = t4
print "Tarefa A concluida."
ENDFORK tarefa_a
FORK_TASK tarefa_b:
```

```
t5 = 200 + 300
resultado_task2 = t5
print "Tarefa B concluida."
ENDFORK tarefa_b
print "Pai aguardando tarefas via Join..."
JOIN_TASKS ALL
print "Resultado Tarefa A: "
print resultado_task1
print "Resultado Tarefa B: "
print resultado_task2
print "--- Teste de Sanidade Fase 1 Finalizado com Sucesso! ---"
```

15.a.2 Geração de código C++ e do executável

MiniPar	C++ gerado
`var x = 42;`	`auto x = 42LL;` (inteiros de 64 bits)
`class C { ... }`, `new C()`, `obj.m()`	`class C { public: ... };`, `std::make_shared<C>()`, `obj->m()`
`par { a; b; }`	uma `std::thread` por comando e `join` de todas
`pure for (...)`	`__minipar_pure_map(n, [&](inicio, fim) { ... })`: iterações divididas entre threads, com escalonamento dinâmico
`impure { ... }`	`std::lock_guard<std::mutex>` sobre o mutex global
`fork t { ... }`	`__active_fork_threads.emplace_back([&]() { ... })`
`fork t @nucleo(n) { ... }`	a mesma thread, que antes chama `__minipar_fixar_nucleo(n)`
`fork t @computador(ip, p) { ... }`	thread que chama `__minipar_executar_remoto(...)`; o corpo é executado pelo computador trabalhador
`join;`	`join()` em todas as threads do pool
`print(a, b)`	`__minipar_print(a, b)`: a linha é escrita de uma vez, sem misturar a saída das threads
`c_channel` / `send` / `receive`	classe `MiniParChannel` (sockets TCP; o socket de escuta fica aberto, para nenhuma mensagem se perder)

Trecho do `main()` gerado para o teste de sanidade:

```
int main(int argc, char** argv) {
    #ifdef _WIN32
        WSADATA wsa;
        WSAStartup(MAKEWORD(2,2), &wsa);
    #endif
    __minipar_iniciar(argc, argv);

    auto soma_compartilhada = 0LL;
    __minipar_print(std::string("--- Iniciando Loop Pure com Regiao Impure ---"));
    {
        // SPar Pure (Map): iteracoes de 'i' divididas entre os nucleos (parallel for)
        const auto __pure_ini0 = 0LL;
        const auto __pure_lim0 = 20LL;
        const long long __pure_n0 = __minipar_pure_count(__pure_ini0, __pure_lim0, 1, false);
        __minipar_pure_map(__pure_n0, [&](long long __pure_a0, long long __pure_b0) {
            for (long long __pure_it0 = __pure_a0; __pure_it0 < __pure_b0; ++__pure_it0) {
                auto i = static_cast<decltype(__pure_ini0)>(__pure_ini0 + __pure_it0 * 1);
                {
                    auto dobro = (i * 2LL);
                    {
                        // SPar Impure Critical Section (Protected by Mutex)
                        std::lock_guard<std::mutex> __impure_guard(__minipar_impure_mtx);
                        soma_compartilhada = (soma_compartilhada + dobro);
                    }
                }
            }
```

```
        });
    }
    __minipar_print(std::string("Resultado da soma apos Pure/Impure: "), soma_compartilhada);
    __minipar_print(std::string("--- Disparando Tarefas Concorrentes com Fork ---"));
    auto resultado_task1 = 0LL;
    auto resultado_task2 = 0LL;
    // Fork Task /* tarefa_a */
    __active_fork_threads.emplace_back([&]() {
        {
            resultado_task1 = (100LL + 50LL);
            __minipar_print(std::string("Tarefa A concluida."));
        }
    });
    // Fork Task /* tarefa_b */
    __active_fork_threads.emplace_back([&]() {
        {
            resultado_task2 = (200LL + 300LL);
            __minipar_print(std::string("Tarefa B concluida."));
        }
    });
...
```

15.b Passo a passo das telas de execução dos programas de teste

1. Selecionar o programa. Na interface didática, o programa é escolhido no seletor "Programa de Teste" e o código aparece no editor. Para programas com `input`, a caixa de entradas mostra as perguntas do programa, uma por linha.

2. Inspecionar o pipeline. O botão "Inspecionar Pipeline" executa as fases do compilador. A aba da AST mostra a árvore; abaixo, a árvore do teste de fork com alocação.

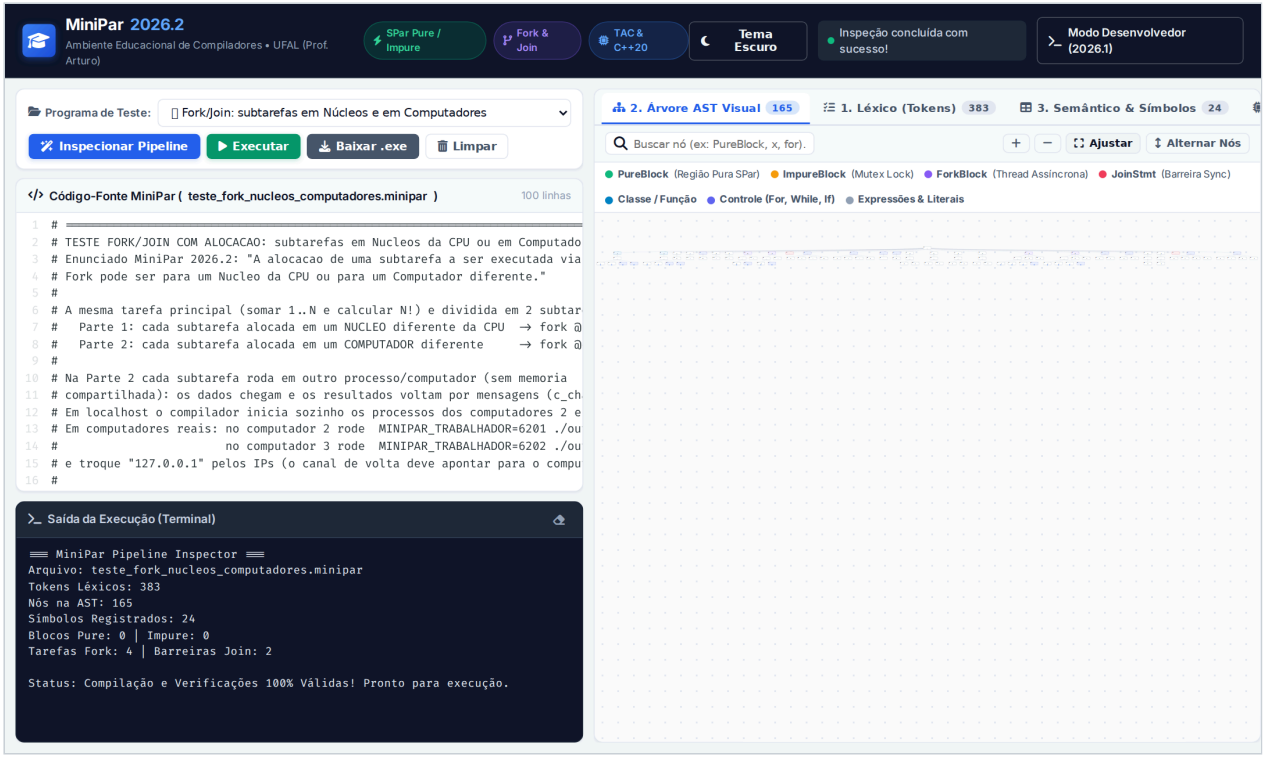


Figura 4 — Árvore AST do teste de fork com alocação em núcleos e computadores.

3. Tokens. Tabela de tokens com categoria, linha e coluna.

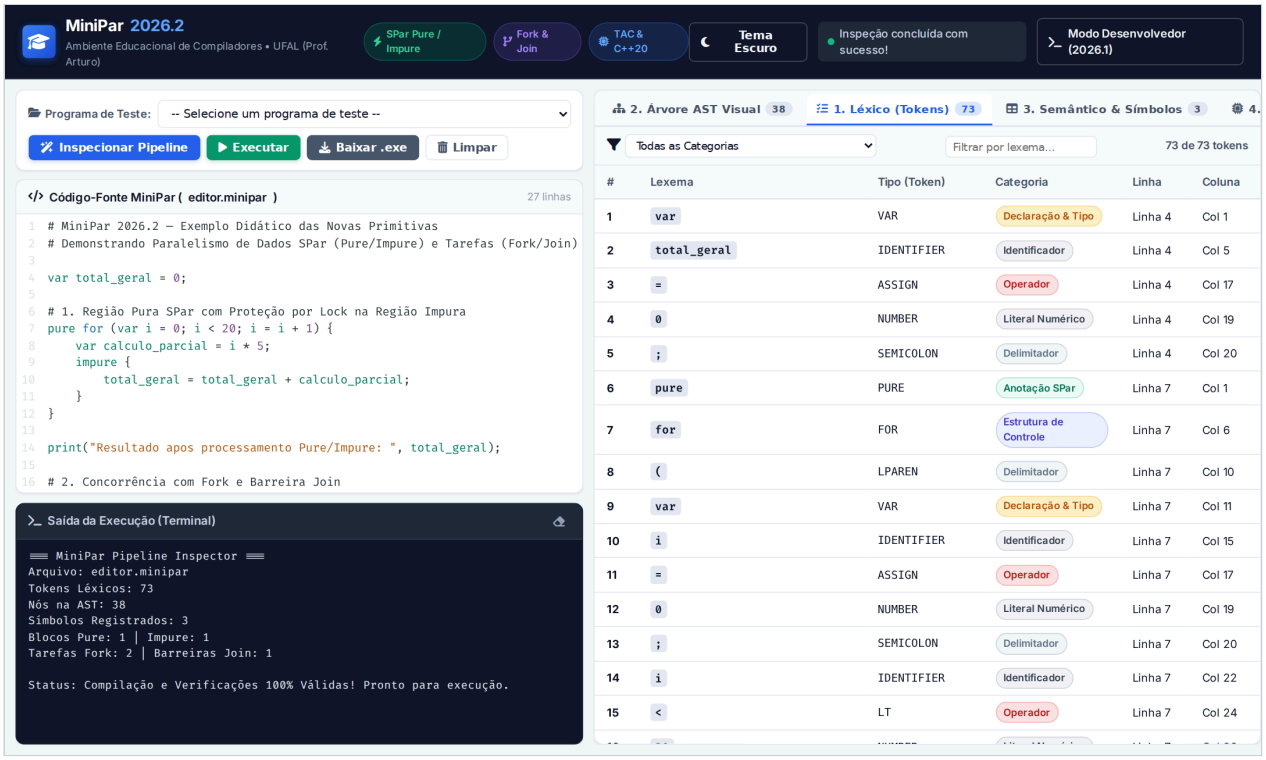


Figura 5 — Aba de tokens (análise léxica).

4. Análise semântica e Tabela de Símbolos.

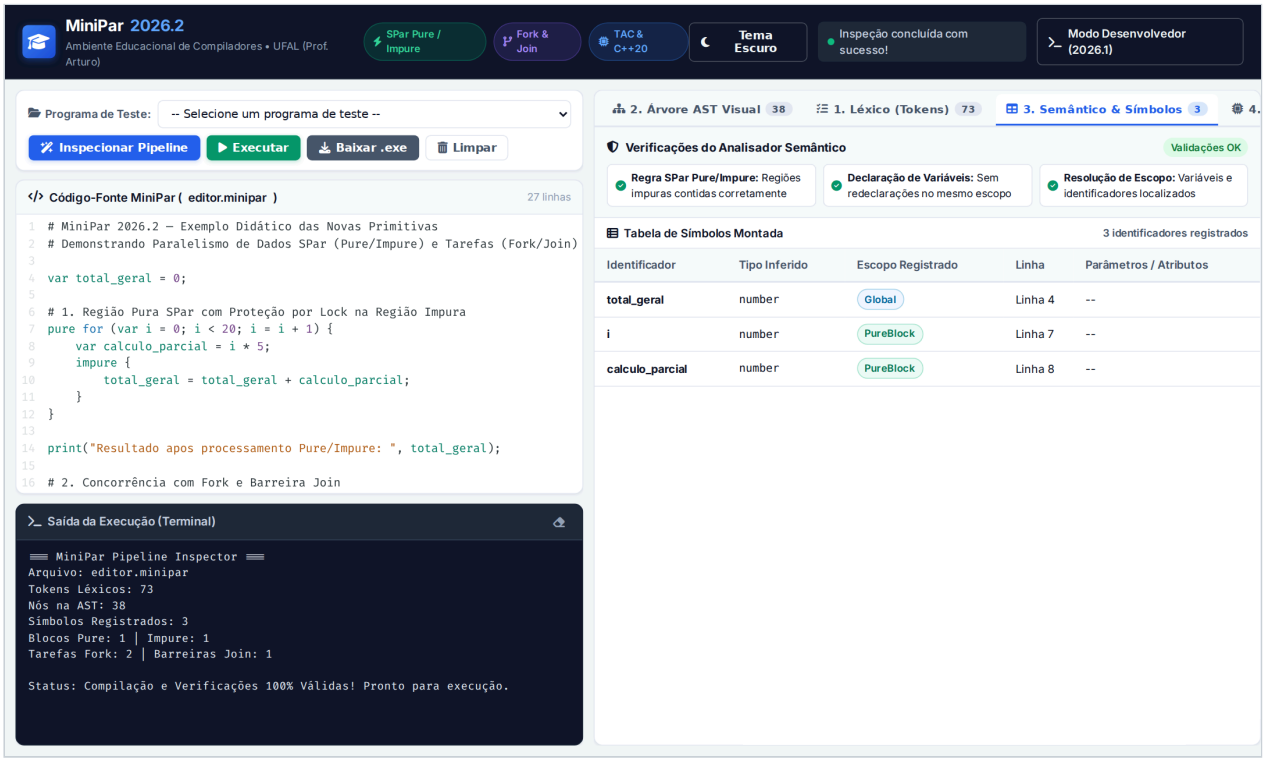


Figura 6 — Aba do semântico: Tabela de Símbolos com escopos e verificações.

5. Código de três endereços e C++ gerado.

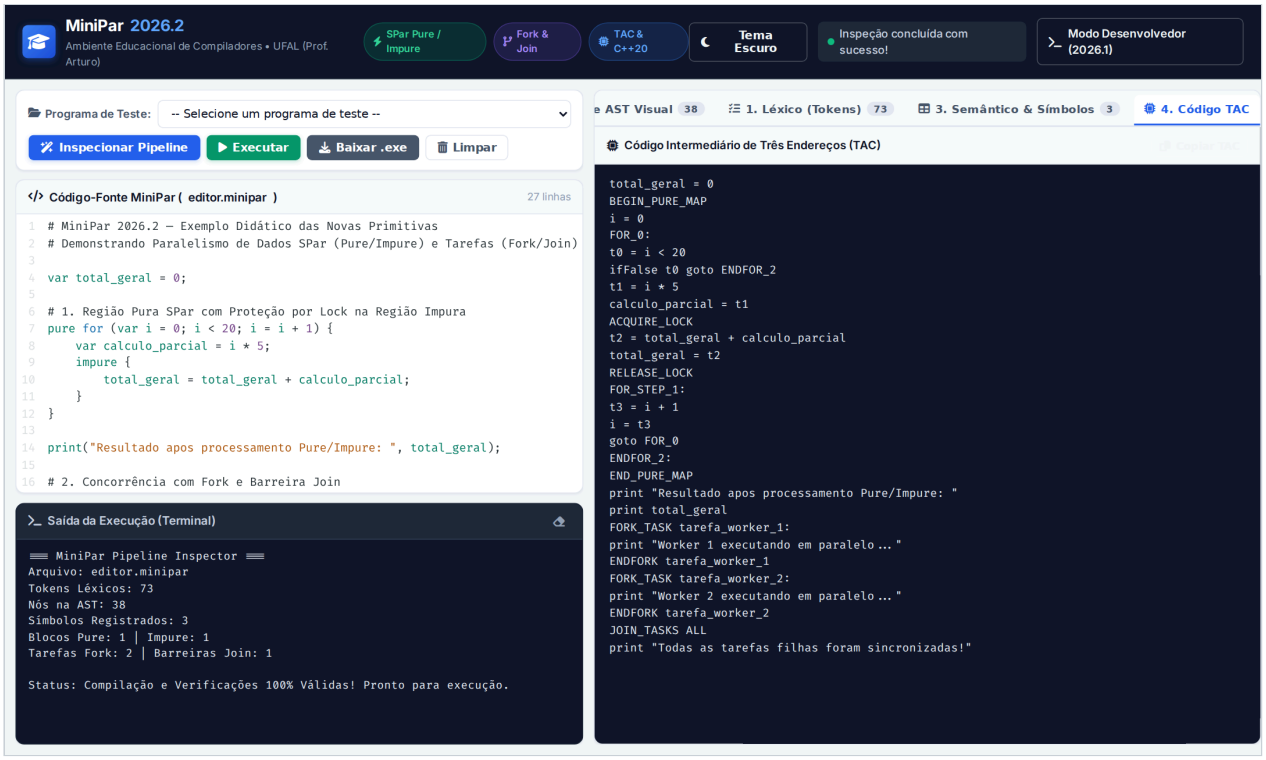


Figura 7 — Aba do TAC.

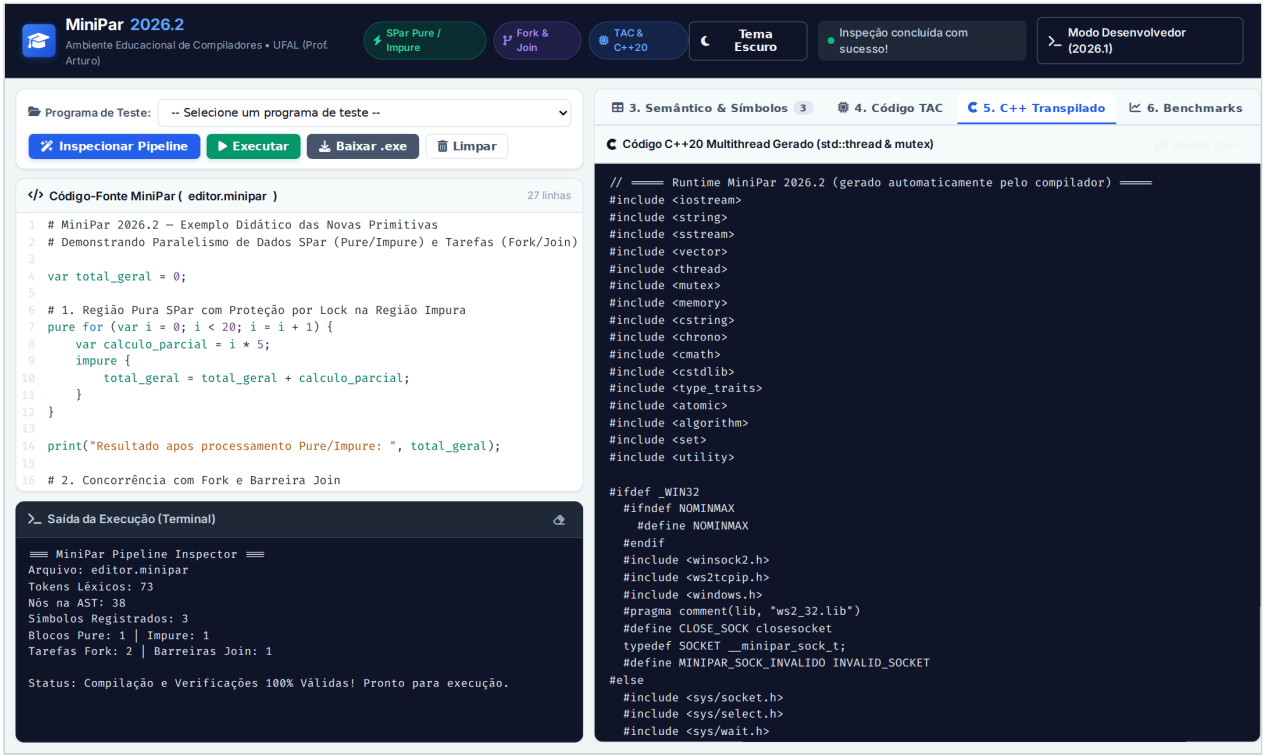


Figura 8 — Aba do C++ gerado.

6. Executar e ver o benchmark. O botão "Executar" compila com o `g++`, executa o binário e mostra a saída no terminal; a aba "Benchmarks" mostra o speedup e os tempos (Teste 9 abaixo).

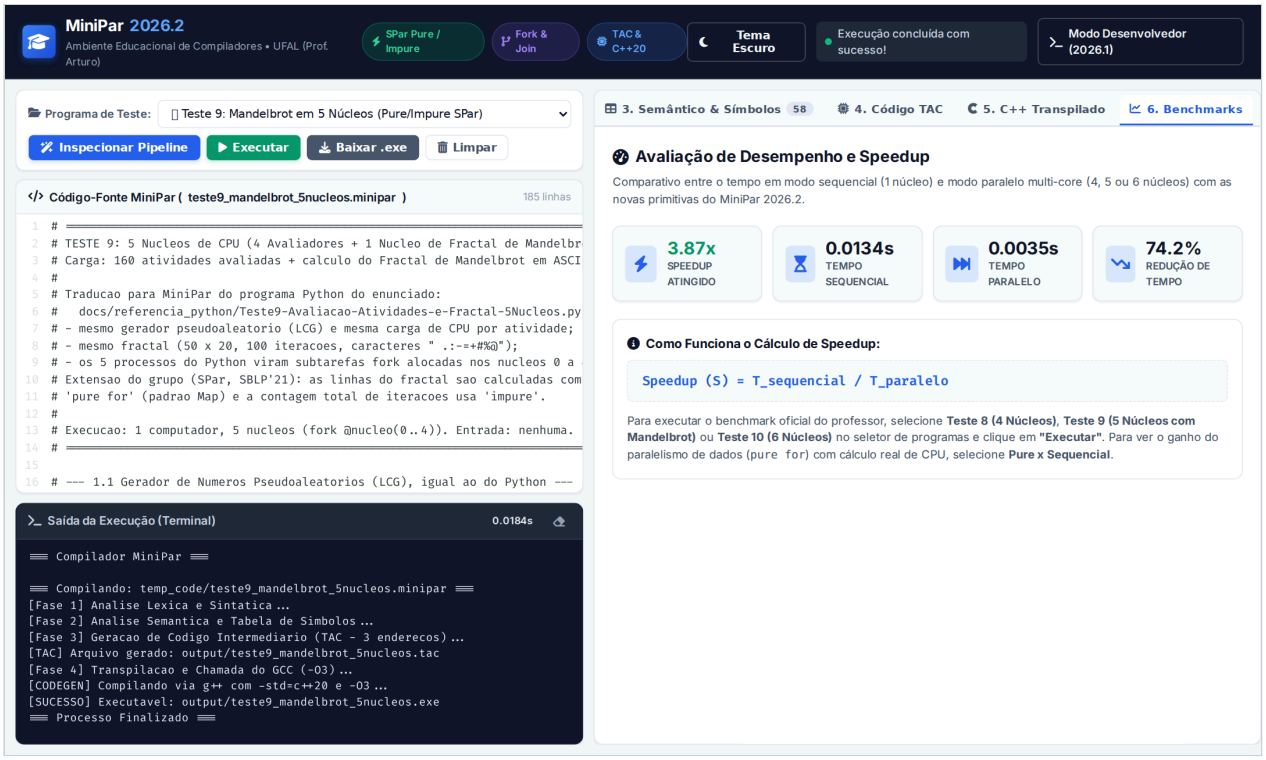


Figura 9 — Execução do Teste 9 com os indicadores de speedup.

7. Programas com entrada. Teste 1 com as entradas `*`, `7` e `6`, uma por linha; o aviso informa que, sem computador extra, o servidor rodou no mesmo computador.

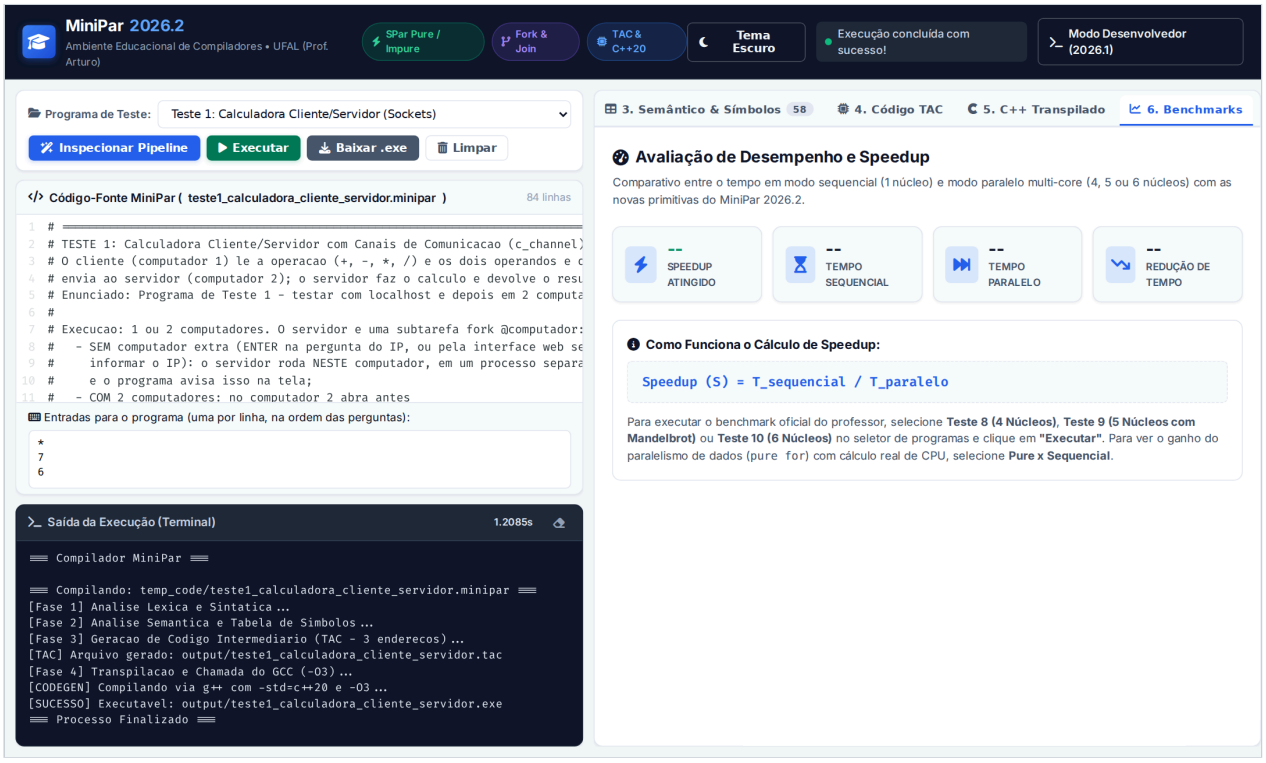


Figura 10 — Teste 1 executado pela interface didática com entradas.

8. Tratamento de erros. Erros sintáticos e semânticos são mostrados com a linha.

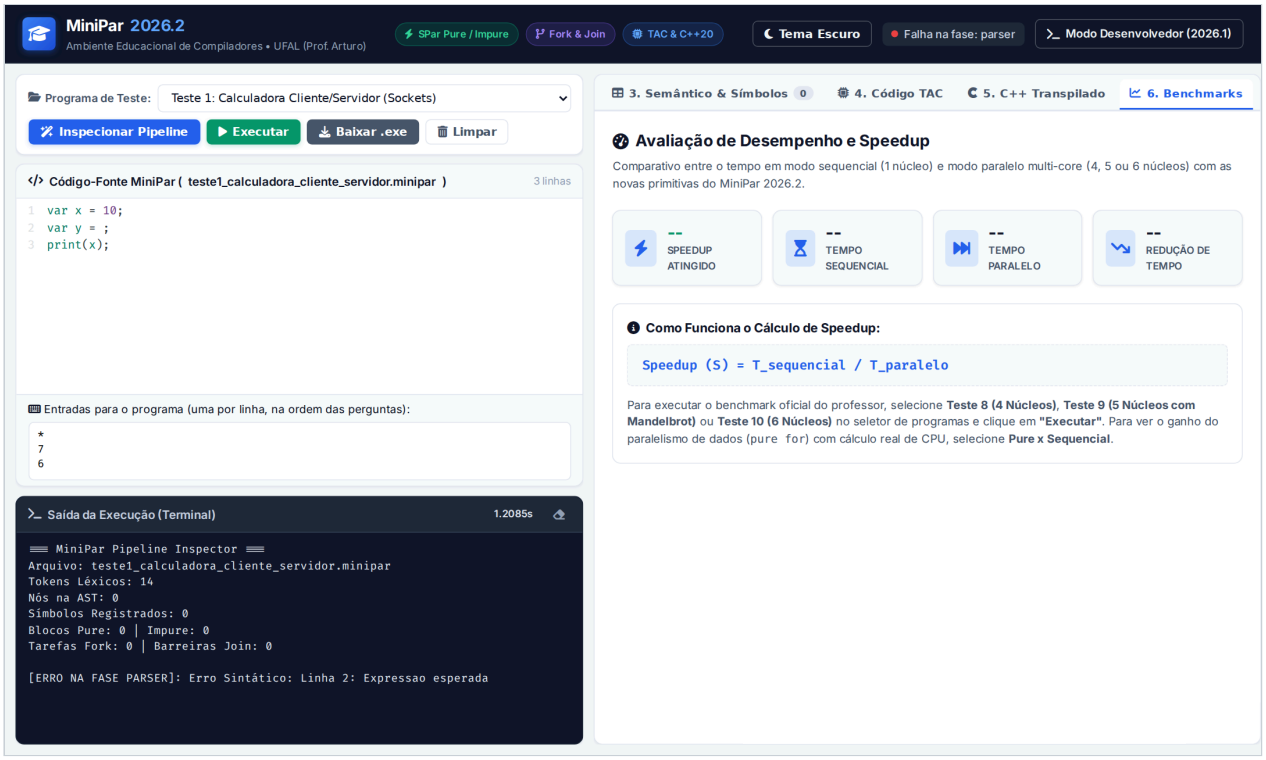


Figura 11 — Erro sintático exibido ao aluno.

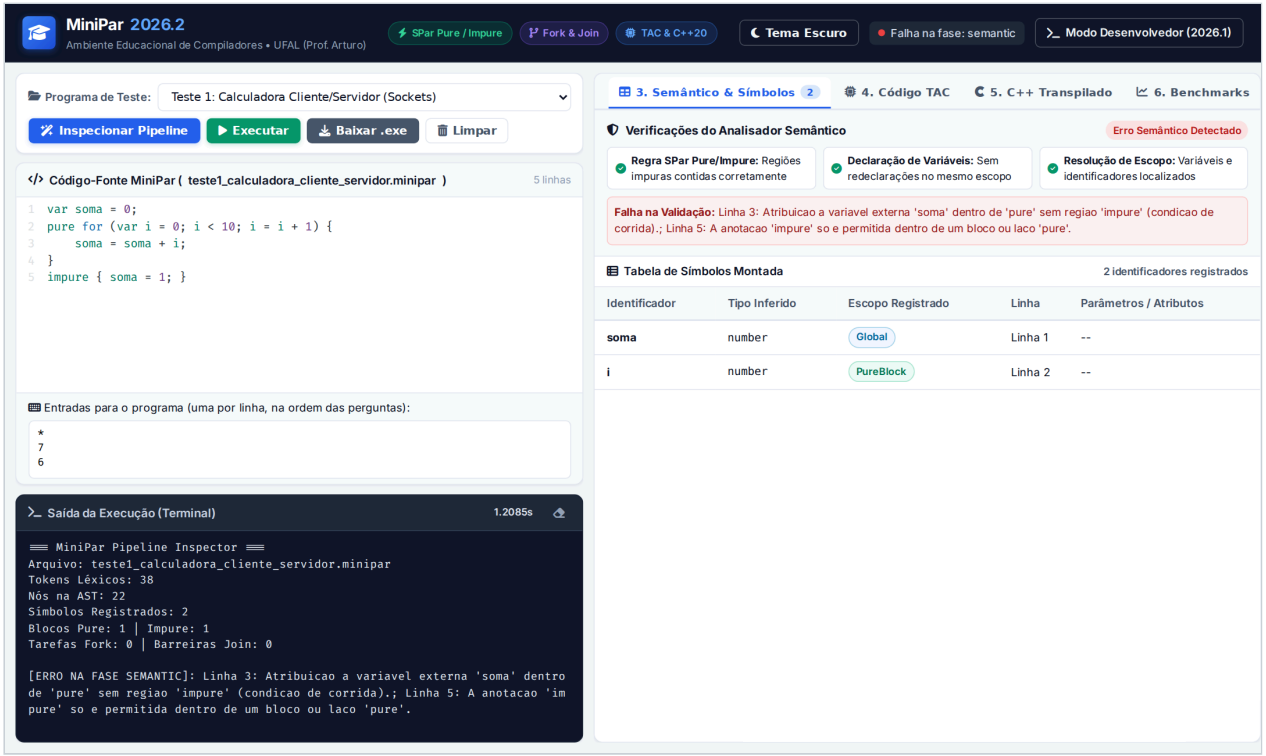


Figura 12 — Erros semânticos das regras de pure/impure.

9. Tema escuro.

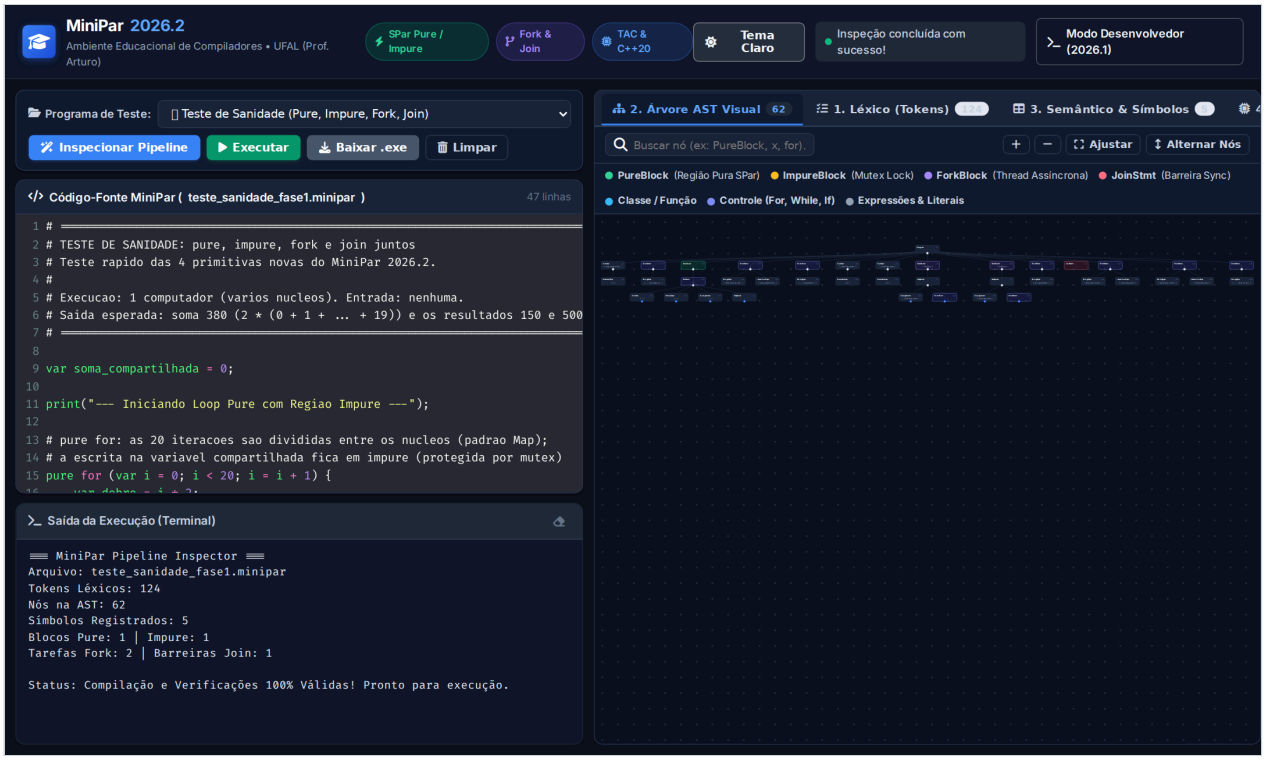


Figura 13 — Interface didática em tema escuro.

10. Execução em vários computadores. O programa `tests/demo_varios_computadores.minipar` pergunta os IPs ao rodar. Em cada computador trabalhador executa-se `MINIPAR_TRABALHADOR=<porta> ./output/demo_varios_computadores.exe`, e no computador principal o programa normalmente. A saída abaixo é da execução em `localhost`; o passo a passo para máquinas reais está em `docs/COMO_DEMONSTRAR_VARIOS_COMPUTADORES.md`.

```
=== Fork/Join com subtarefas em computadores diferentes ===
IP deste computador (PC1) [ENTER = 127.0.0.1]: IP do computador 2 (soma) [ENTER = 127.0.0.1]: IP
do computador 3 (fatorial) [ENTER = 127.0.0.1]: [fork subtarefa_soma] Nenhum computador extra
aberto em 127.0.0.1:6201 -> a subtarefa vai rodar NESTE computador, em um processo separado.
[fork subtarefa_fatorial] Nenhum computador extra aberto em 127.0.0.1:6202 -> a subtarefa vai
rodar NESTE computador, em um processo separado.
[computador :6201] trabalhador pronto, aguardando subtarefas...
[computador :6202] trabalhador pronto, aguardando subtarefas...
[fork subtarefa_soma] subtarefa enviada ao computador 127.0.0.1:6201
[fork subtarefa_fatorial] subtarefa enviada ao computador 127.0.0.1:6202
[computador :6201] executando subtarefa subtarefa_soma
[computador :6202] executando subtarefa subtarefa_fatorial
Recebi n = 10000 do computador 127.0.0.1 - calculando a soma de 1 a n...
Recebi n = 12 do computador 127.0.0.1 - calculando o fatorial de n...
[fork subtarefa_soma] subtarefa concluida no computador 127.0.0.1:6201
[fork subtarefa_fatorial] subtarefa concluida no computador 127.0.0.1:6202
Soma de 1 a 10000 calculada no computador 127.0.0.1 = 50005000
Fatorial de 12 calculado no computador 127.0.0.1 = 479001600
=== Fim: as duas subtarefas foram executadas em outros computadores ===
```

15.c Limitações conhecidas

- O compilador foi desenvolvido e testado em Linux/WSL2; o runtime tem os trechos específicos para Windows, mas eles não foram testados em Windows nativo.
- Um `fork` dentro de uma função cujo `join` fica fora dela não é seguro, pois a subtarefa acessa por referência as variáveis locais da função.

- O Teste 2 é executado em um computador (duas threads); a execução em dois computadores está demonstrada no Teste 1 e na demonstração de vários computadores.
- Listas literais são numéricas; para textos usa-se `matrix`.

16. CÓDIGO-FONTE, PROGRAMAS DE TESTE E RELATÓRIO NO GITHUB

O código-fonte do compilador, os programas de teste (`tests/`), os testes automatizados, a documentação (`README.md`, `BNF-grammar.md`, `docs/`) e este relatório (`relatorio/`) estão no repositório:

https://github.com/ludsno/minipar-2026-2

17. LINK DO CÓDIGO-FONTE NA CAPA

O link do repositório consta na capa deste relatório.

CONCLUSÃO

O MiniPar 2026.2 atende ao enunciado: mantém as interfaces de terminal e Web de 2026.1 e acrescenta a interface Web didática com a árvore AST gráfica; estende a gramática com `fork` e `join`, com alocação da subtarefa em núcleo ou em outro computador; implementa os Testes 8, 9 e 10 a partir dos programas Python fornecidos; e modela o compilador com Componentes de Software, com as interfaces presentes também no código. O pedido específico ao grupo — as anotações `pure` e `impure` do modelo SPar, testadas com o Mandelbrot — foi implementado com paralelismo de dados real: o teste Pure × Sequencial mostra speedup de cerca de 10x com 16 threads e de 1x quando o processo é restrito a um núcleo.

A execução dos 18 programas e dos 33 testes automatizados confirma o funcionamento integrado dos componentes. Como trabalhos futuros, ficam a validação em Windows nativo, o `join` automático para `fork` dentro de funções e a recuperação de erros sintáticos sem interromper a análise no primeiro erro.

REFERÊNCIAS

Materiais da disciplina

1. DOMINGUEZ, Arturo Hernandez. *Projeto do Compilador (Orientado a Objetos) MiniPar 2026.2, com testes solicitados e nova segunda interface Web*. Material da disciplina de Compiladores. Maceió: UFAL, Instituto de Computação, 2026.
2. DOMINGUEZ, Arturo Hernandez. *Elementos de Implementação Utilizando Componentes de Software*. Material de apoio da disciplina de Compiladores. Maceió: UFAL, Instituto de Computação.
3. LIMA, Emanuele V. de J.; QUINTELA, José Lucas de O.; BOMFIM, Luiz Miguel de M. *Relatório de Projeto: MiniPar-Improvement (MiniPar 2026.1)*. Maceió: UFAL, 2026. Código em: https://github.com/luizwhirl/minipar-improvement.
4. AHO, Alfred V.; LAM, Monica S.; SETHI, Ravi; ULLMAN, Jeffrey D. *Compiladores: princípios, técnicas e ferramentas*. 2. ed. São Paulo: Pearson Addison-Wesley, 2008.

Referências adicionais

5. LÖFF, Júnior; HOFFMANN, Renato B.; GRIEBLER, Dalvan; FERNANDES, Luiz G. *High-Level Stream and Data Parallelism in C++ for Multi-Cores*. In: Simpósio Brasileiro de Linguagens de Programação (SBLP), 2021.
6. INSTITUTO FEDERAL DE SANTA CATARINA (IFSC). *Compilador Educacional Completo*. Disponível em: http://pesquisa06.lages.ifsc.edu.br/. Acesso em: nov. 2026.
7. LOUDEN, Kenneth C. *Compiladores: princípios e práticas*. São Paulo: Pioneira Thomson Learning, 2004.
8. ISO/IEC. *ISO/IEC 14882:2020: Programming Languages — C++*. Genebra: ISO, 2020.
9. PALLETS PROJECTS. *Flask Documentation (3.0.x)*. Disponível em: https://flask.palletsprojects.com/. Acesso em: nov. 2026.
10. BOSTOCK, Mike. *D3.js — Data-Driven Documents*. Disponível em: https://d3js.org/. Acesso em: nov. 2026.
11. HAVERBEKE, Marijn. *CodeMirror: in-browser code editor*. Disponível em: https://codemirror.net/. Acesso em: nov. 2026.
12. PYTHON SOFTWARE FOUNDATION. *The Python 3 Language Reference*. Disponível em: https://docs.python.org/3/reference/. Acesso em: nov. 2026.